

Aplikace neuronových sítí

Učení bez učitele a generativní modely

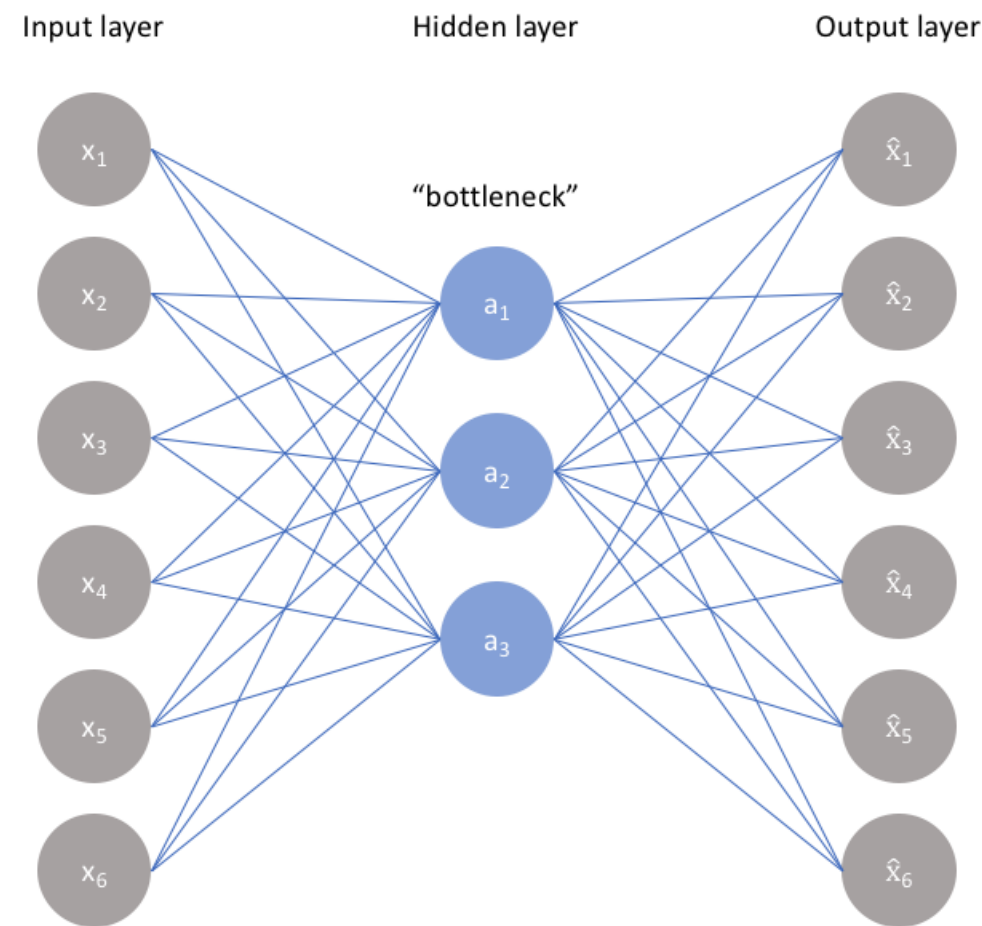
Autoenkodér

- úkolem rekonstrukce vstupních dat
- architektura typu encoder - dekodér
- vstupní vektor/obrázek je redukován enkodérem na nějaký *bottleneck*
- poté znovu rekonstruován dekodérem
- lze nahlížet jako na kompresi
- cílem co nejmenší rekonstrukční odchylka, např. mean square error (MSE)

$$L(x, x') = \frac{1}{D} \|x - x'\|^2$$

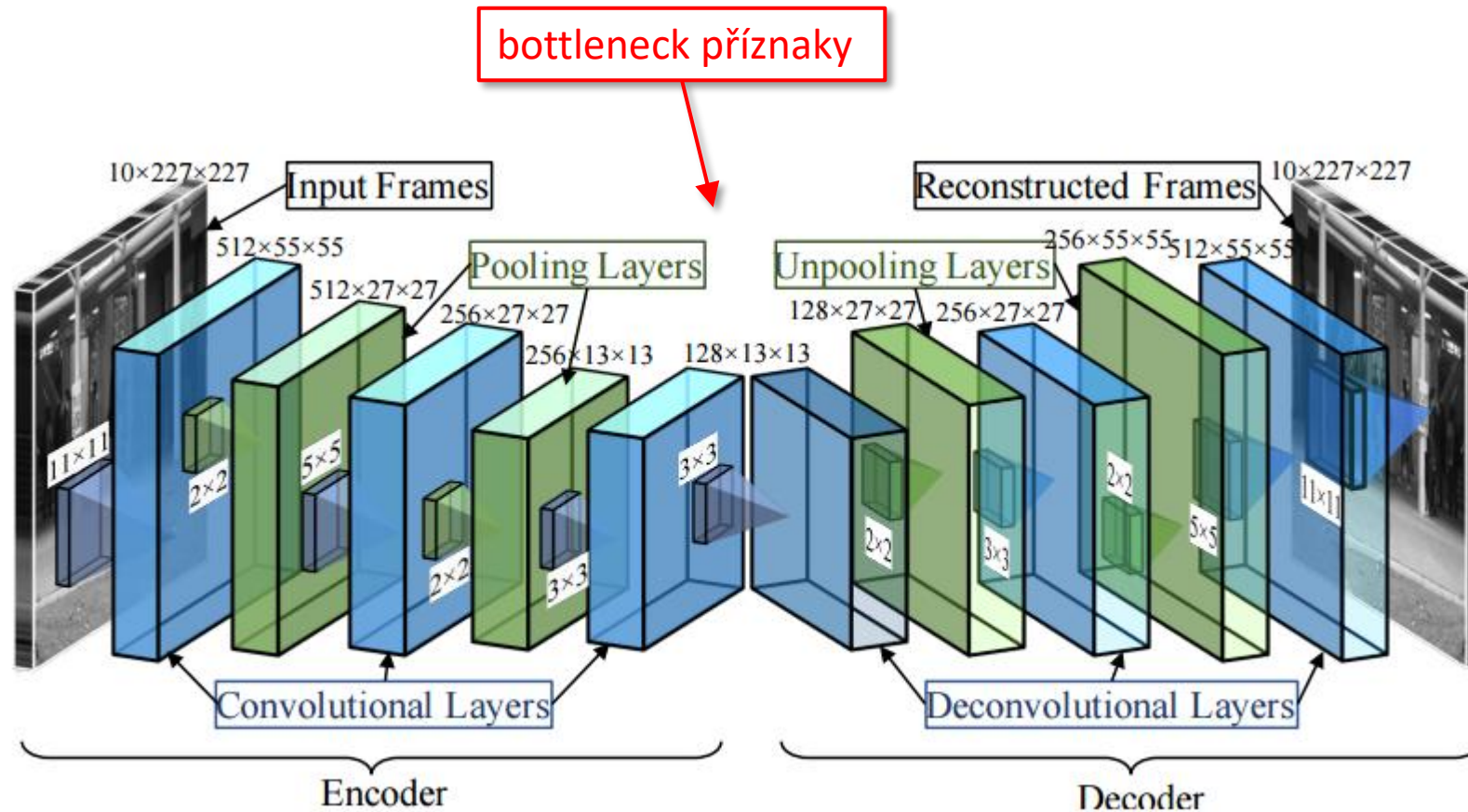
kde D je délka vektoru x

- Spec. případ s jednou skrytou vrstvou a bez nelinearity
 $\approx$ PCA
 - nejsou to samé, ale výsledné váhy pokrývají stejný prostor
 - PCA má navíc omezení jejich vzájemné ortogonality



obrázek: <https://www.jeremyjordan.me/autoencoders/>

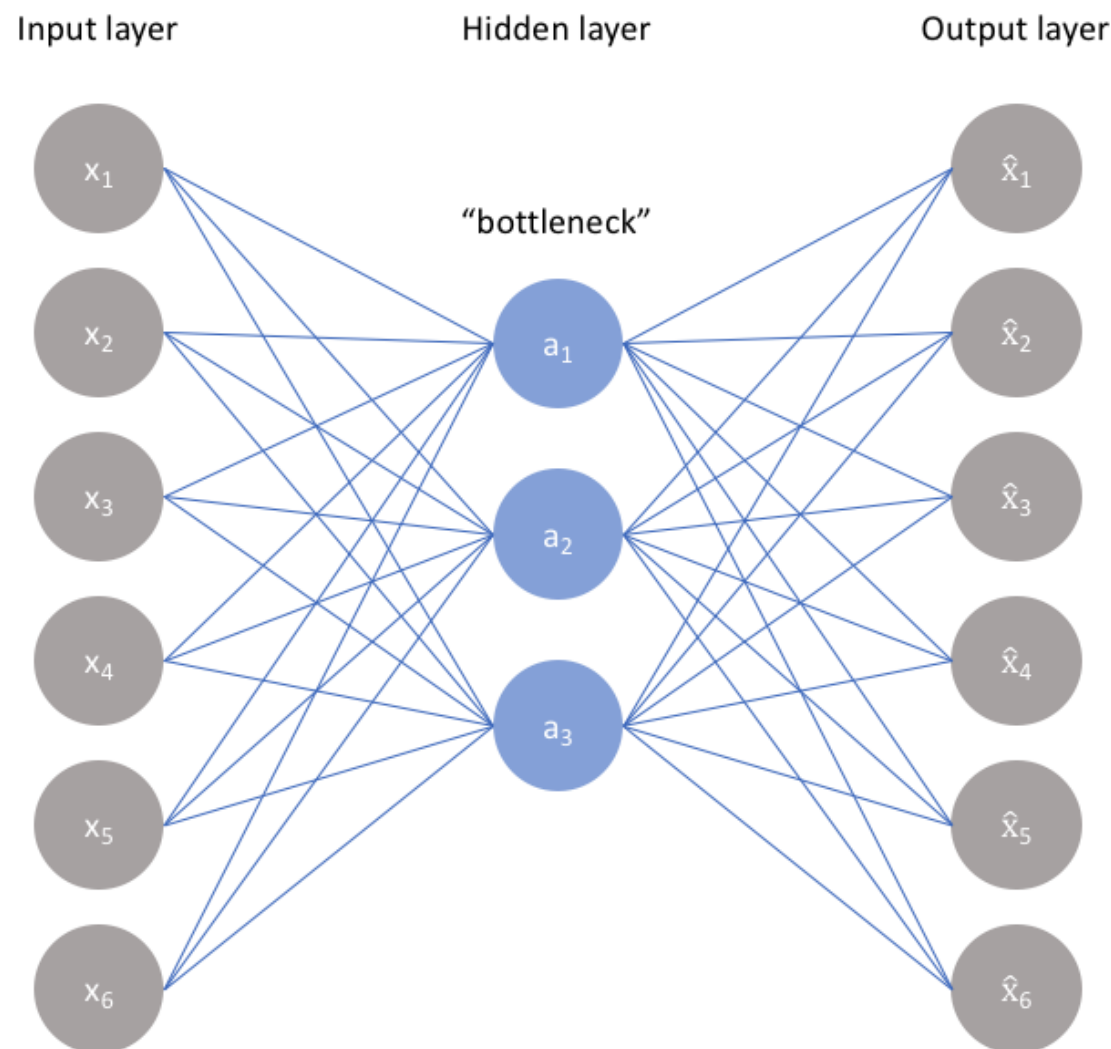
Konvoluční autoenkodér



- nahrazuje fully connected vrstvy konvolučními

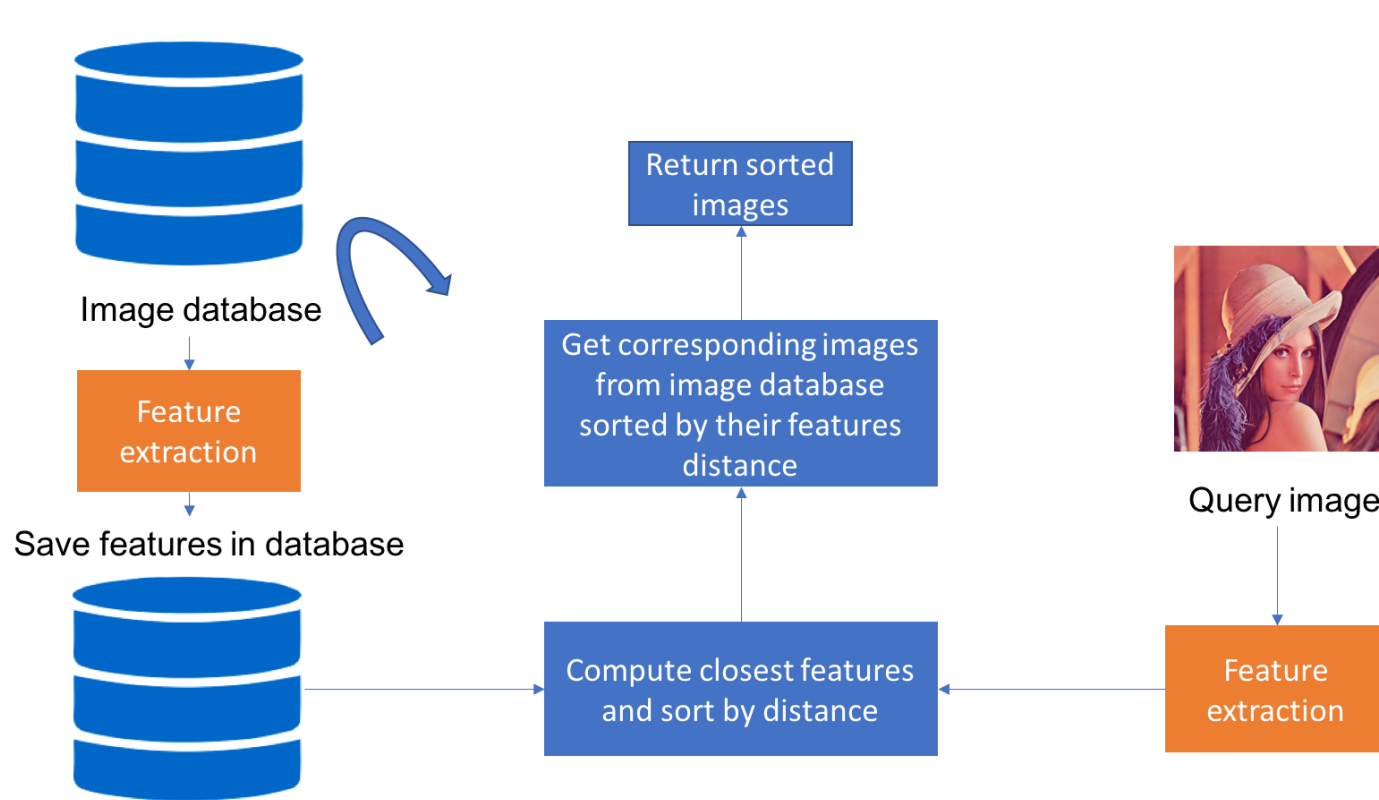
Autoenkodér pro detekci anomálií

- myšlenka: natrénujeme na “čistých” datech
- v testovací fázi:
 - známá data, která odpovídají trénovacím → malá rekonstrukční odchylka
 - neznámá data (anomálie) → velká odchylka
- V literatuře jako anomaly či outlier detection
- lze využít např. pro detekci nečistot v materiálu apod.

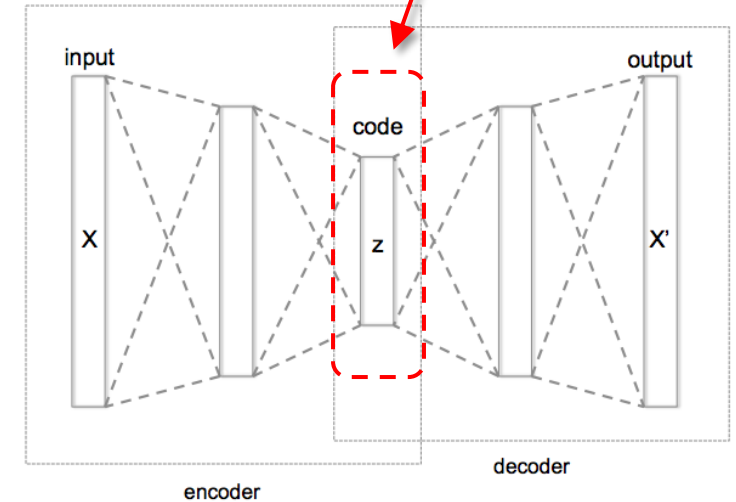


Content-based image retrieval

- myšlenka: autoenkodér natrénovaný na velkém množství obrázků použít pro extrakci příznaků



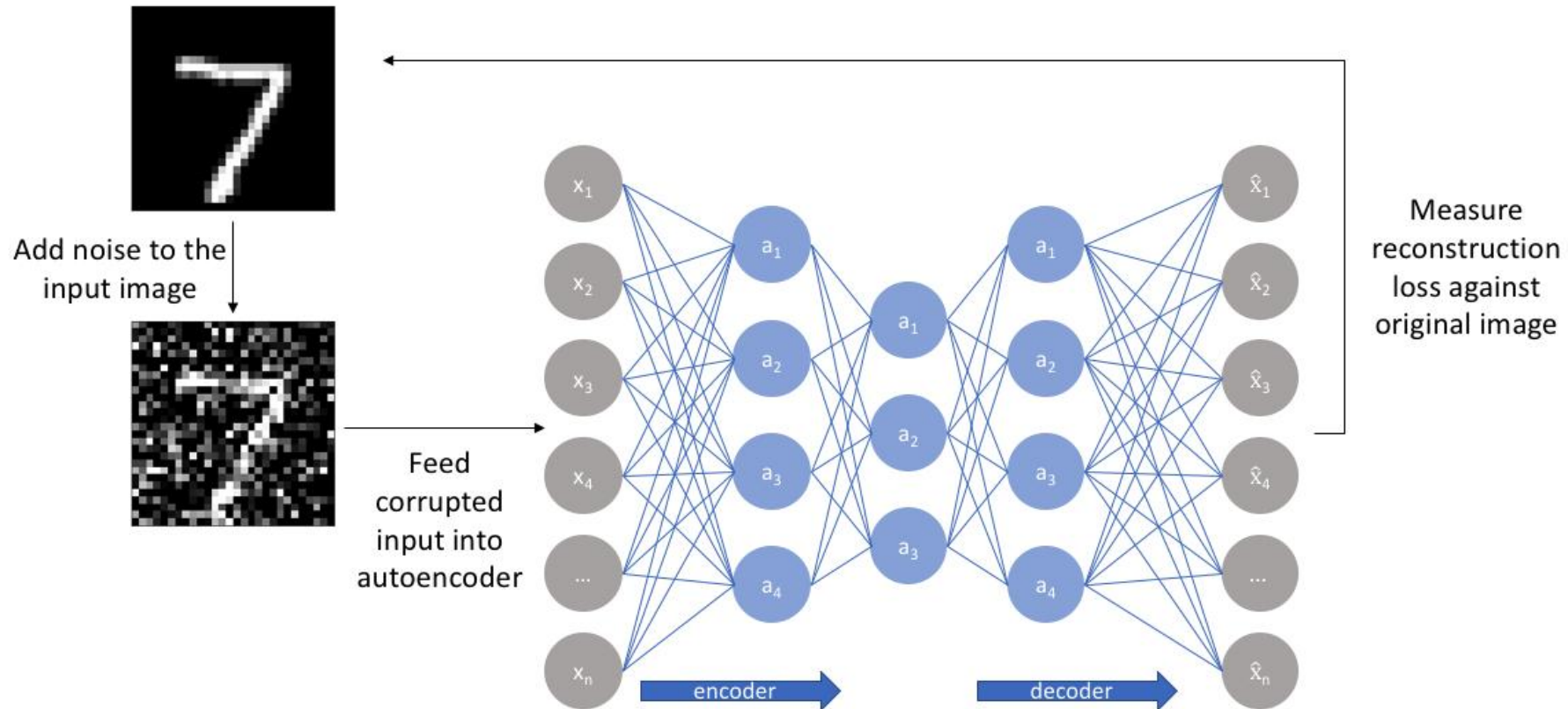
bottleneck vector jako příznaky



- nejpodobnější obrázek vyhledáme např. metodou nejblížšího souseda nad extrahovanými příznaky

obrázek: <https://blog.sicara.com/keras-tutorial-content-based-image-retrieval-convolutional-denoising-autoencoder-dc91450cc511>

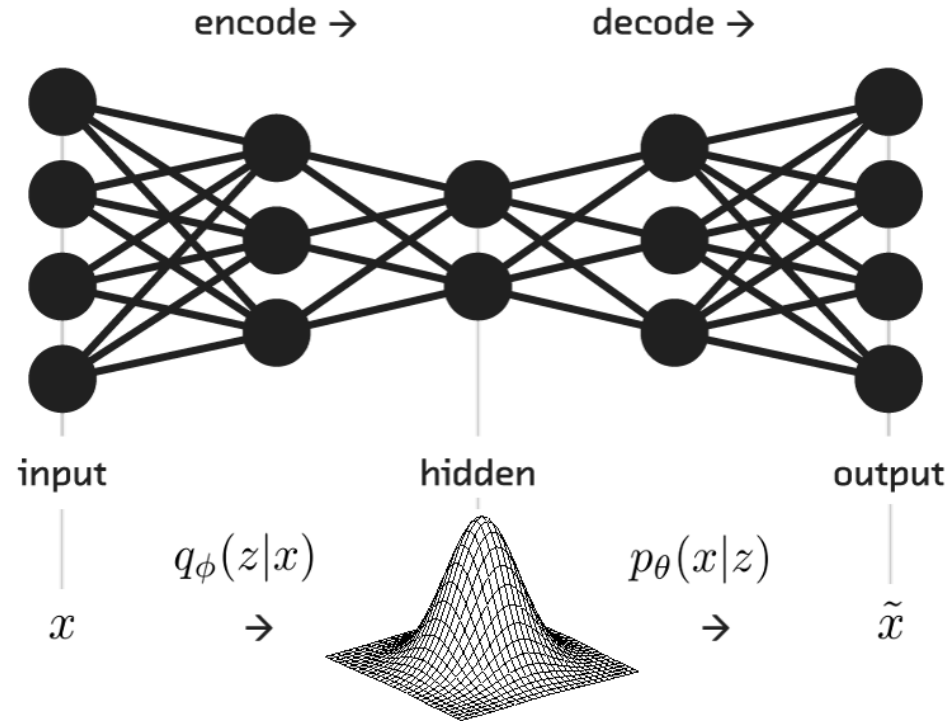
Denoising autoencoder



- vstup je zašuměný → síť rekonstruuje čistý obrázek
- natrénovaný autoenkodér odstraní šum
- lze využít i pro inpainting apod.
- princip formulace podobný jako u colorization, super resolution, ...

obrázek: <https://www.jeremyjordan.me/autoencoders/>

Variační autoenkodér



celkový loss

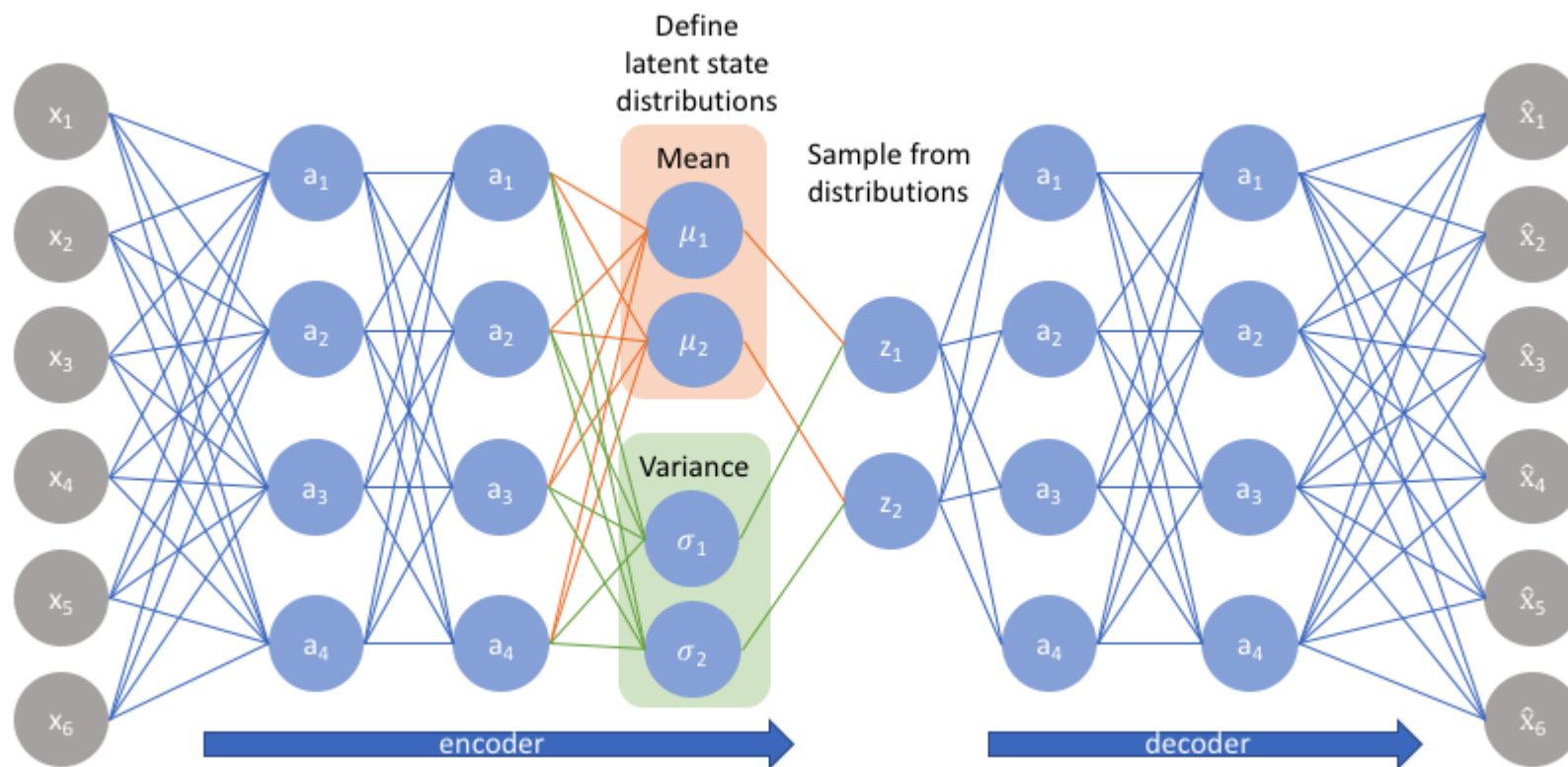
$$\mathcal{L}(x, \hat{x}) + \sum_j KL(q_j(z|x) || p(z))$$

Kullback-Liebler divergence

- omezuje bottleneck příznaky na gaussové (normální) rozložení
- kromě rekonstrukční odchylky navíc [Kullback-Liebler divergence](#) jako kritérium na bottleneck

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)} \quad \dots \text{měří podobnost, na kolik je bottleneck } (Q) \text{ gaussovský } (P)$$

Implementace variačního autoenkodéru

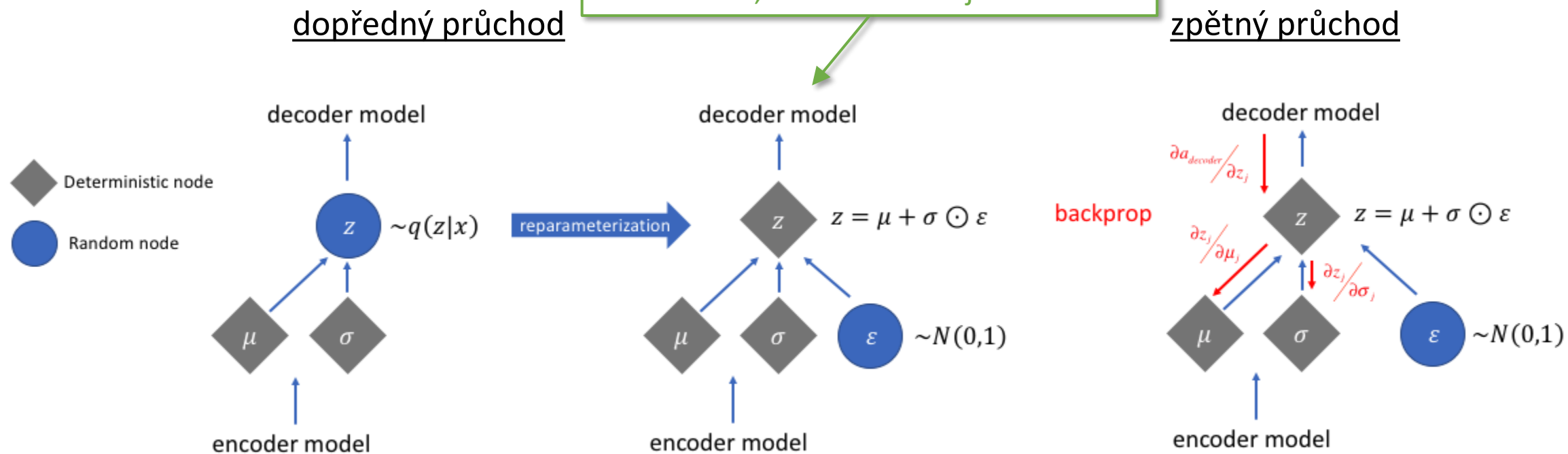


- KL nelze v praxi efektivně vyhodnotit, je to střední hodnota (integrál) přes všechny možné hodnoty
- namísto síť vygeneruje dva vektory průměru μ a rozptylu σ^2 gaussovského rozložení
- pak se využije následujícího:

$$-\mathcal{D}_{KL}(q_{\phi}(z|x)||p_{\theta}(z)) = \frac{1}{2} \sum (1 + \log(\sigma^2) - \mu^2 - \sigma^2)$$

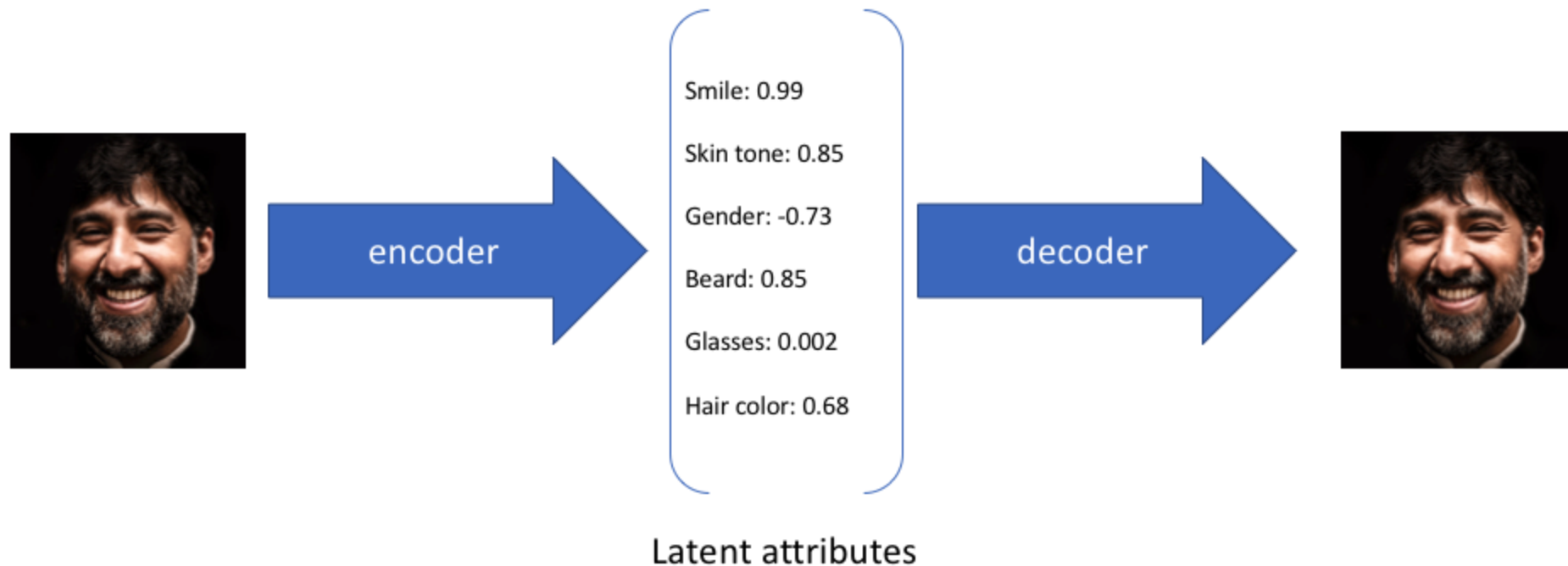
Reparametrizační trik

po reparametrizaci zůstává vzorkování z ekvivalentní, ale zároveň už jde derivovat



- vzorkuje se vždy ze standardního $N(0, 1)$ gaussovského rozložení s nulovým průměrem a jednotkovým rozptylem
- průměr a rozptyl se řeší afinní transformací (škálování = std. odchylka, posun = průměr) vzorkovaných dat

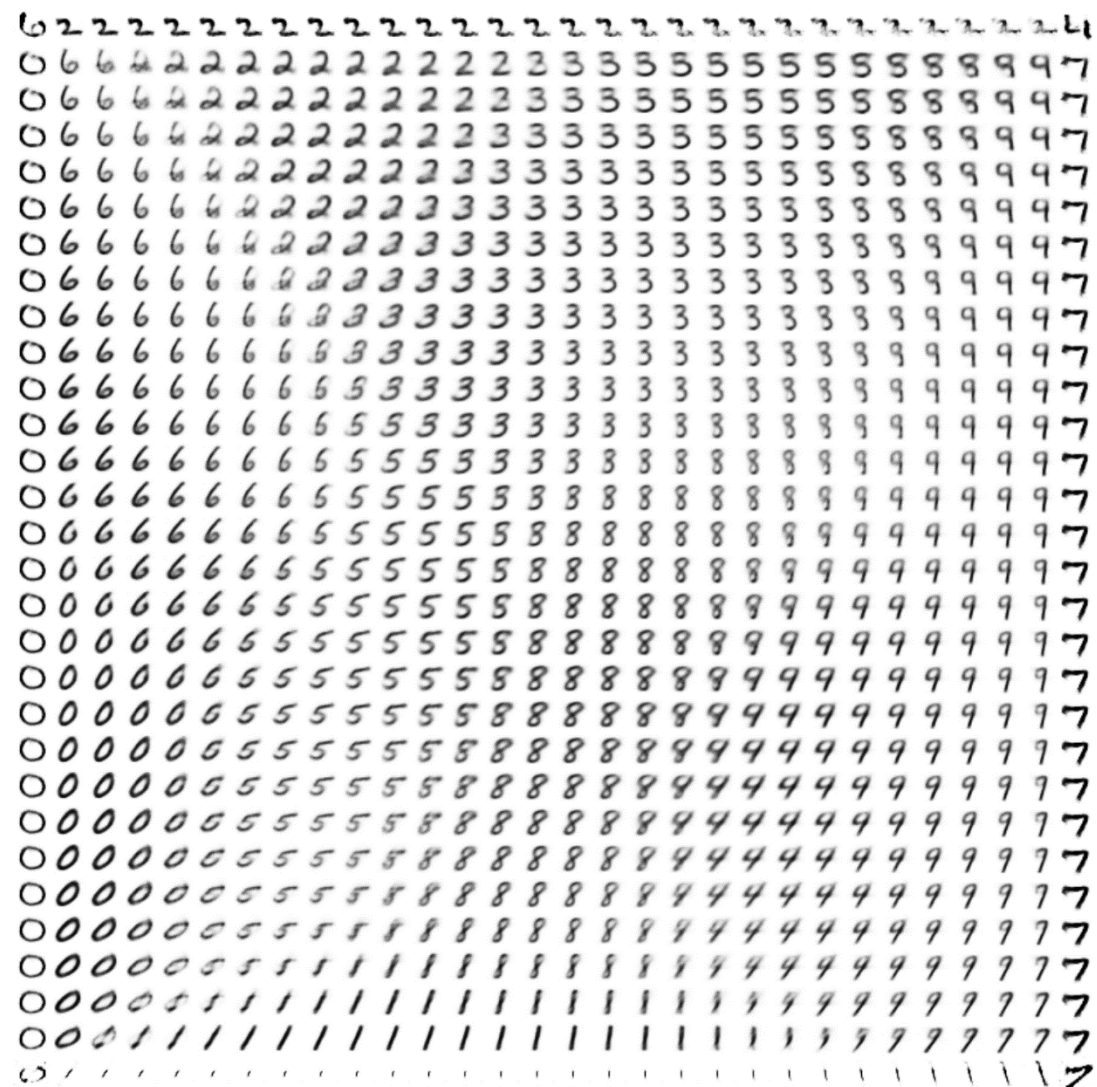
Intepretace latentních “bottleneck” parametrů



- na bottleneck vektor lze nahlížet jako na skryté parametry
- samplováním parametrů z normálního rozložení lze inicializovat dekodér a generovat různé obrázky
- variační autoenkodér tedy patří mezi generativní modely

Skryté parametry variačního autoenkodéru

- na obrázku příklad dvourozměrného normálního rozložení (dva skryté parametry)
- pohybem v tomto 2D prostoru generujeme různé MNIST číslice



obrázek: <http://blog.fastforwardlabs.com/2016/08/22/under-the-hood-of-the-variational-autoencoder-in.html>

Skryté parametry variačního autoenkodéru



Obrázky vygenerované variačním autoenkodérem



Labeled faces in the wild



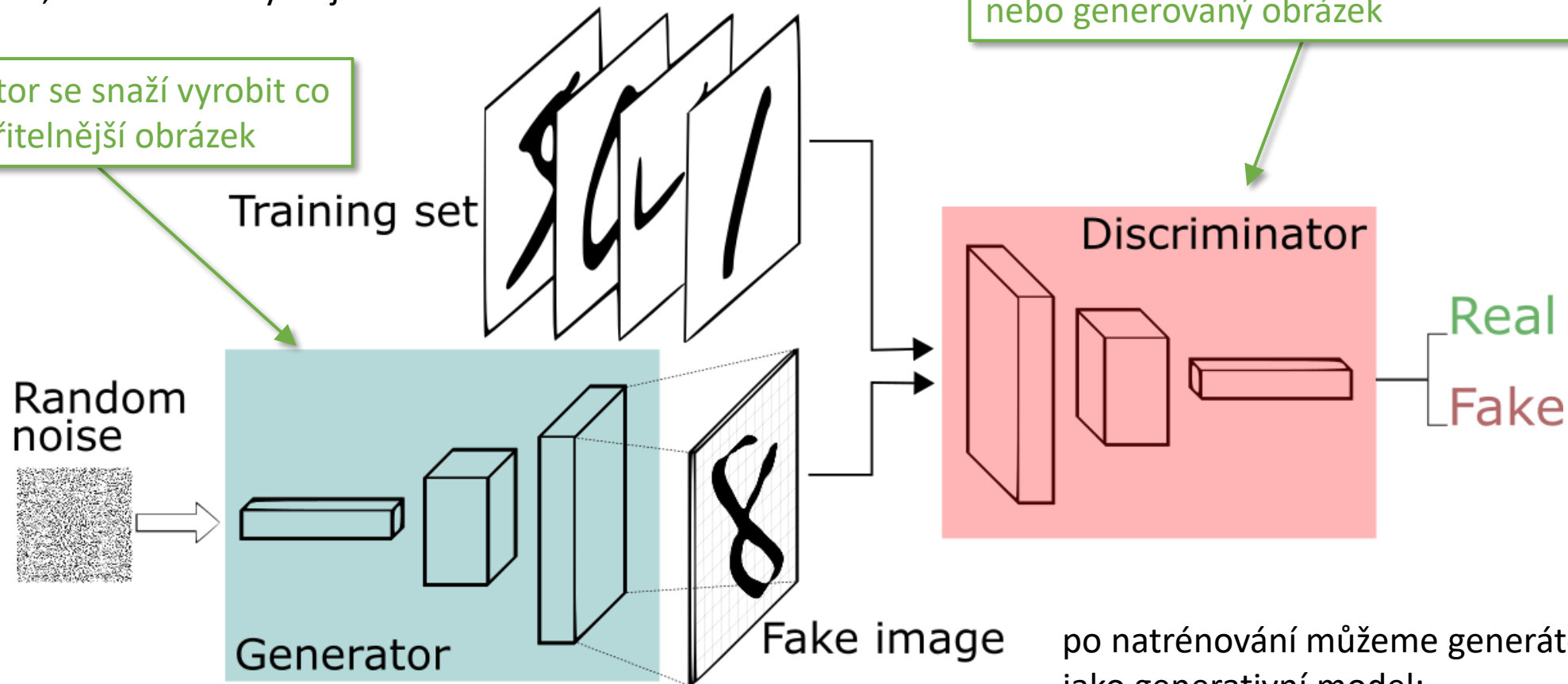
CIFAR-10

obrázek: <https://jaan.io/what-is-variational-autoencoder-vae-tutorial/>

Generative adversarial network (GAN)

- [Goodfellow et al.: Generative Adversarial Networks](#)
- učení dvou sítí zároveň: generátor vs diskriminátor
- “soutěž”, která z nich vyhraje

generátor se snaží vyrobit co nejuvěřitelnější obrázek



po natrénování můžeme generátor použít
jako generativní model:
random inicializace → generovaný obrázek

Kritérium generativní adversarial sítě

celkový loss je

nezávislé na generátoru →
generátor lze updatovat odděleně

$$L_{\text{GAN}}(G, D) = \underbrace{\mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)]}_{D(x) \dots \text{výstup diskriminátoru}} + \mathbb{E}_{z \sim p_z(z)} \left[\log \left(1 - D(G(z)) \right) \right]$$

$D(x)$... výstup diskriminátoru

$G(z)$... výstup generátoru

rozdíl oproti klasické optimlizaci:

- loss $L_{\text{GAN}}(G, D)$ **maximalizujeme** přes diskriminátor tak, aby $D(x)$ bylo 1 pro **reálné** obrázky, 0 pro fake
- generátor se naopak snaží, aby $D(G(z))$ bylo 1 pro **falešné** obrázky → přes generátor **minimalizujeme**

Trénování generativní adversarial sítě

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by **ascending** its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

update diskriminátoru
generátor je zafixovaný

gradient na diskriminátor

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by **descending** its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

update generátoru
diskriminátor je zafixovaný

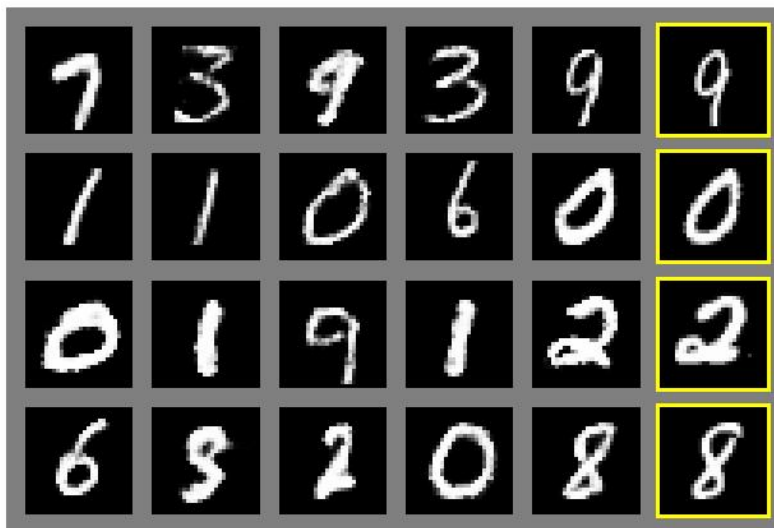
gradient na generátor

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

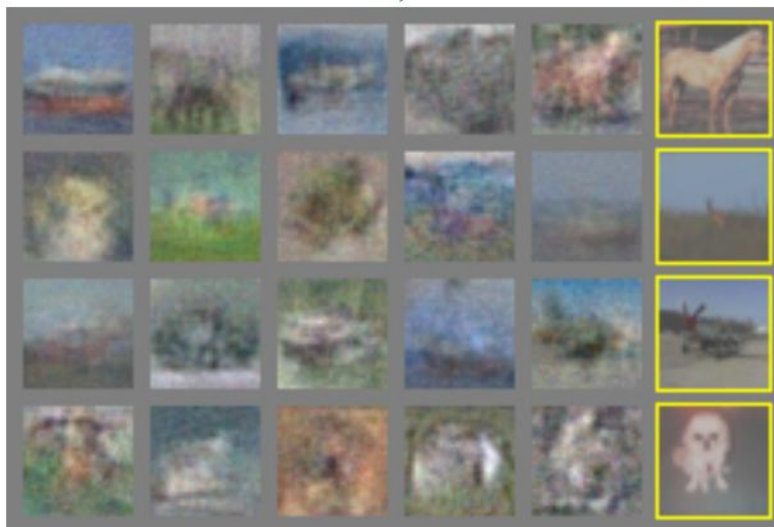
GAN ukázky v původním článku

MNIST



a)

CIFAR



c)



TFD

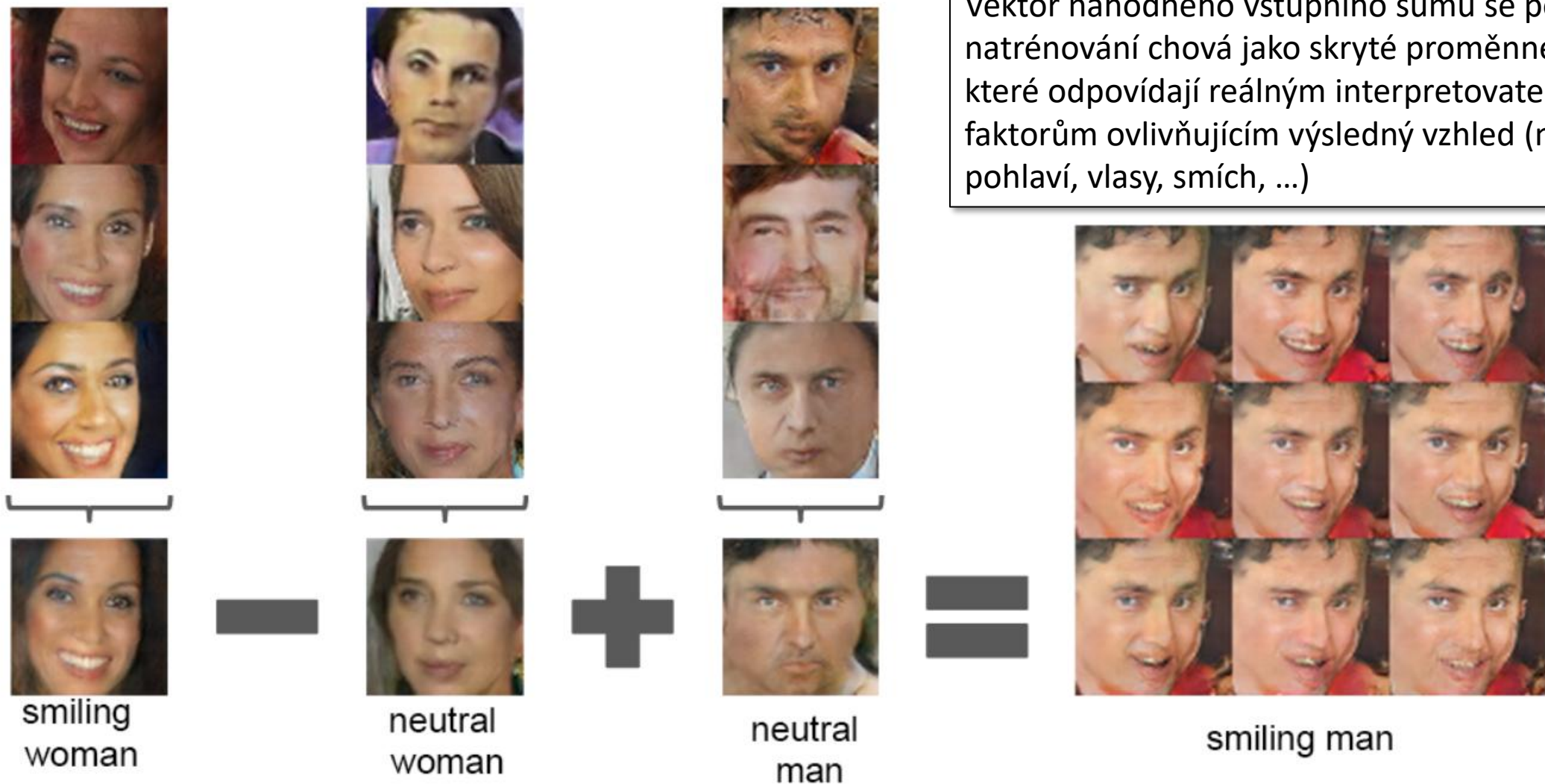
b)



d)

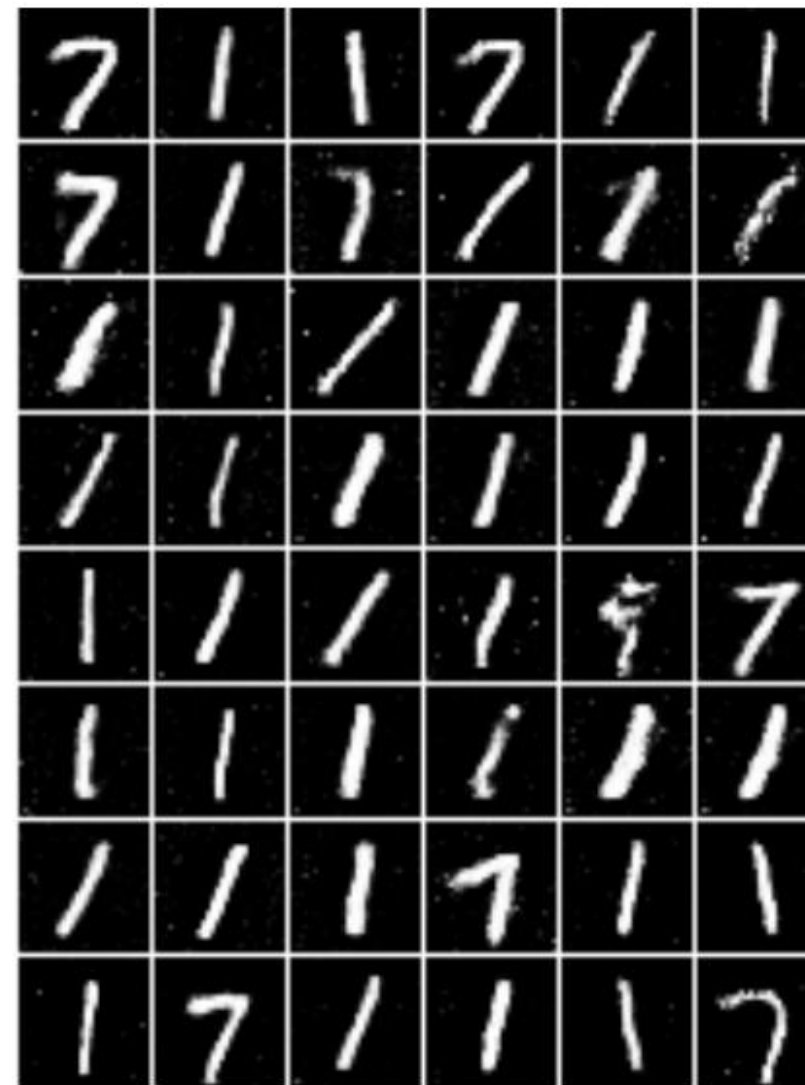
zdroj: [Goodfellow et al.: Generative Adversarial Networks](#)

Vektorová aritmetika



Typické problémy při trénování GAN

- Mizející gradient (vanishing gradients)
 - Pokud diskriminátor je příliš dobrý
 - Pravděpodobnosti $D(G(z))$ jsou pak příliš blízko 0 nebo 1 → viz sigmoid gradient
- Mode collapse
 - Některé např. číslice se generují snáze
 - Generátor to zjistí a jiné přestane produkovat
 - Diskriminátor se chytne do lokálního minima



Wasserstein GAN (2017)

- [Arjovsky et al: Wasserstein GAN](#)
- Namísto $\min/\max D(z)$ a $D(G(z))$ optimalizuje
$$\mathbb{E}_x[D(x)] - \mathbb{E}_z[D(G(z))]$$
- Maximalizace rozdílu skóre pro reálná vs fake data
- Výstup není pravděpodobnost, ale skóre $(-\infty, +\infty)$

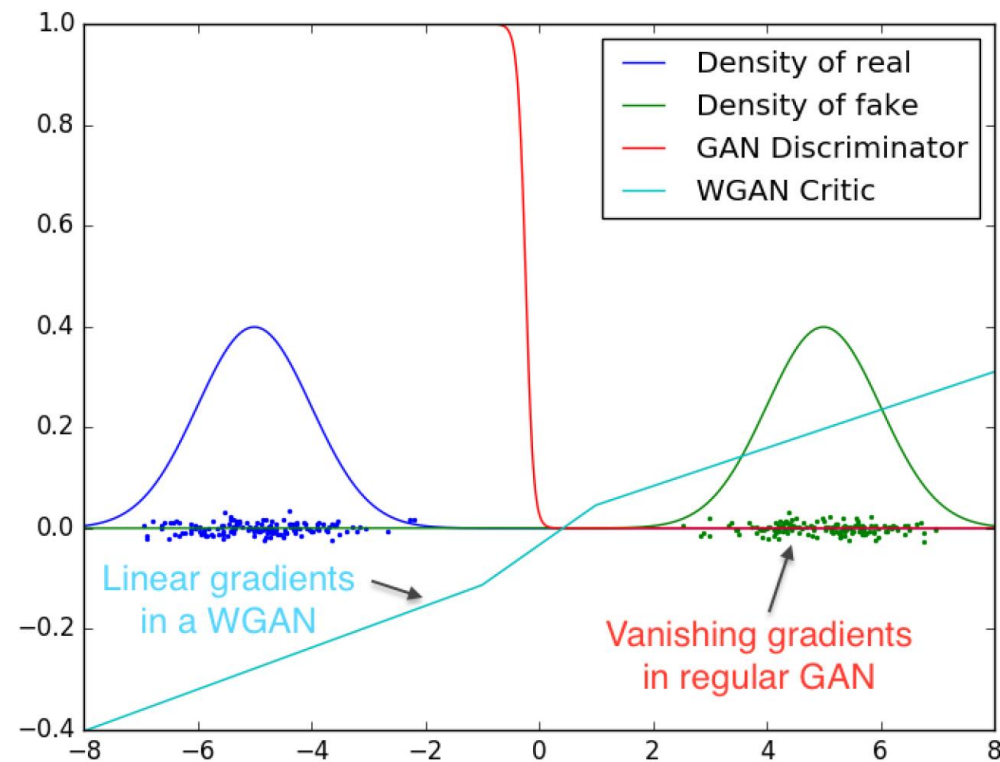


Figure 2: Optimal discriminator and critic when learning to differentiate two Gaussians. As we can see, the discriminator of a minimax GAN saturates and results in vanishing gradients. Our WGAN critic provides very clean gradients on all parts of the space.

pix2pix (2016)

- [Isola et al: Image-to-Image Translation with Conditional Adversarial Networks](#)
- Unifikovaný framework pro úlohy typu $výstupní_obrázek = funkce(vstupní_obrázek)$
- **Není unsupervised**: dataset jsou páry obrázků (vstup, výstup)

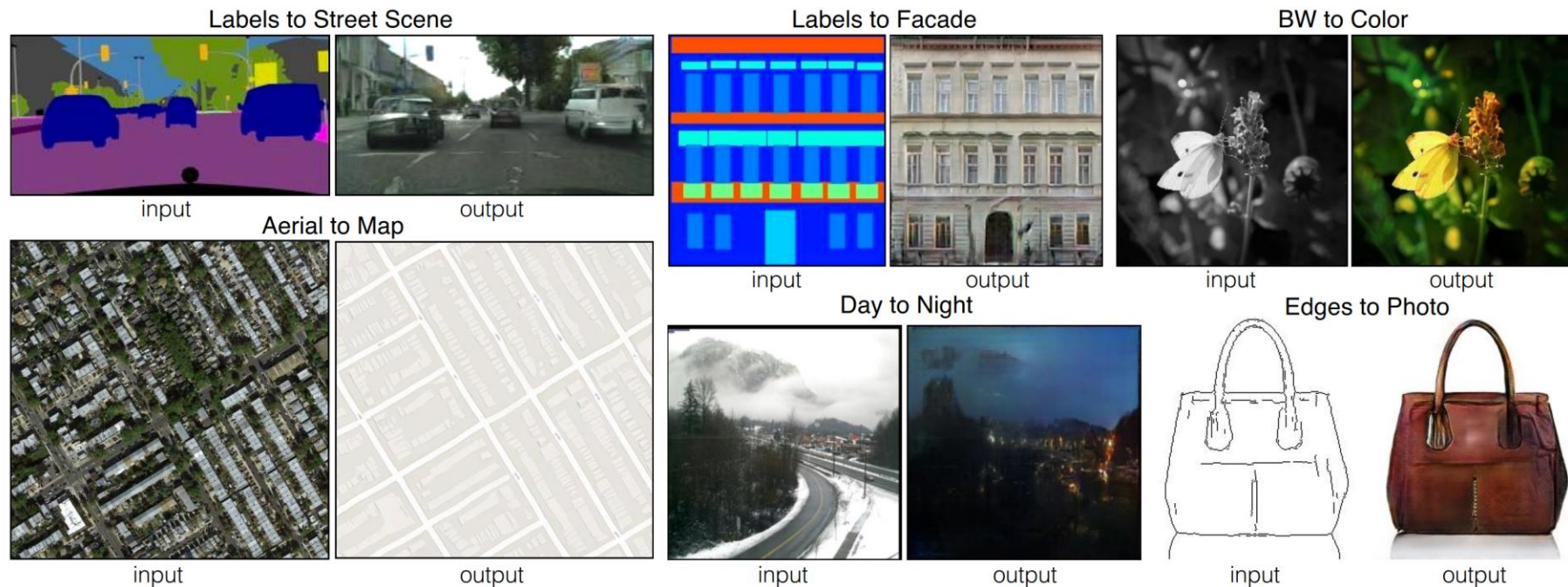
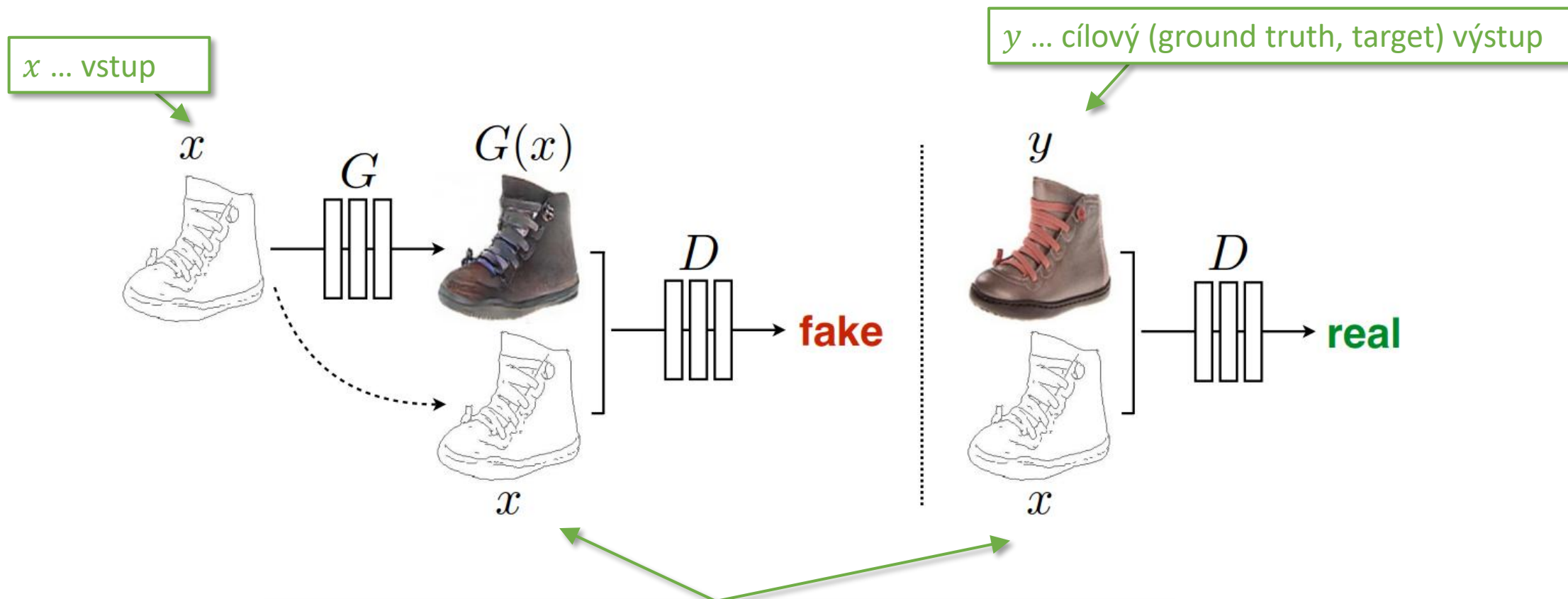


Figure 1: Many problems in image processing, graphics, and vision involve translating an input image into a corresponding output image. These problems are often treated with application-specific algorithms, even though the setting is always the same: map pixels to pixels. Conditional adversarial nets are a general-purpose solution that appears to work well on a wide variety of these problems. Here we show results of the method on several. In each case we use the same architecture and objective, and simply train on different data.

pix2pix (2016)



diskriminátor $D(x, y)$ přejímá kromě jedné z možností $G(x)$ vs y i samotný vstup $x \rightarrow$ „kontroluje“, jak dobře výstup odpovídá vstupu $\rightarrow$ diskriminátor se chová jako trénovatelná loss funkce

pix2pix (2016)

- Conditional GAN (cGAN)

- generátor i diskriminátor jsou podmíněny („vidí“) vstupem x , které není $\sim \mathcal{N}(0,1)$
- nepodmíněná GAN: generátor generuje náhodně (vstupem je random z), diskriminátor vidí jen $G(z)$

- GAN kritérium pix2pix je

$$L_{cGAN}(G, D) = \mathbb{E}_{x,y}[\log D(x, y)] + \mathbb{E}_{x,z} \left[\log \left(1 - D(x, G(x, z)) \right) \right]$$

- Zároveň ale chce, aby výstup generátoru byl co nejblíže ground truth y , tj.

$$L_{L1}(G) = \mathbb{E}_{x,z} [\|y - G(x, z)\|_1]$$

- Celková loss funkce tedy je

$$G^* = \arg \min_G \max_D L_{cGAN}(G, D) + \lambda L_{L1}(G)$$

pix2pix generátor

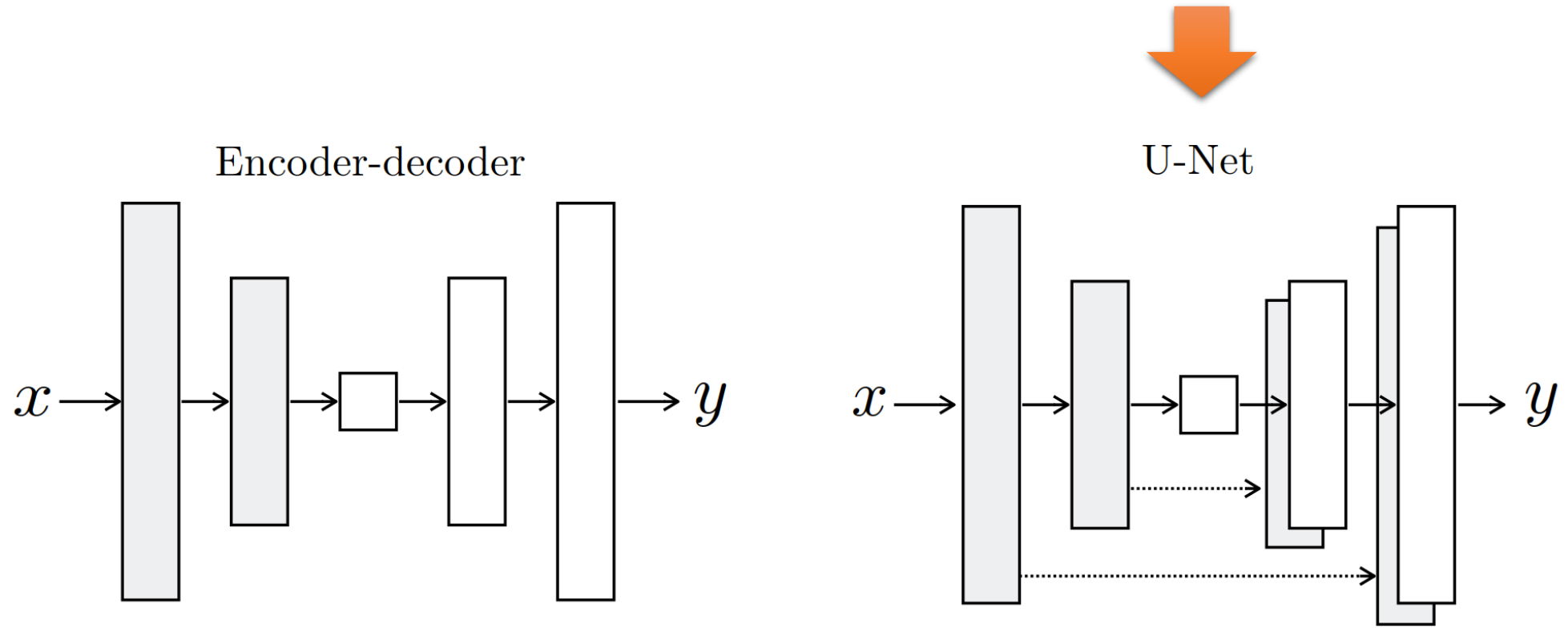
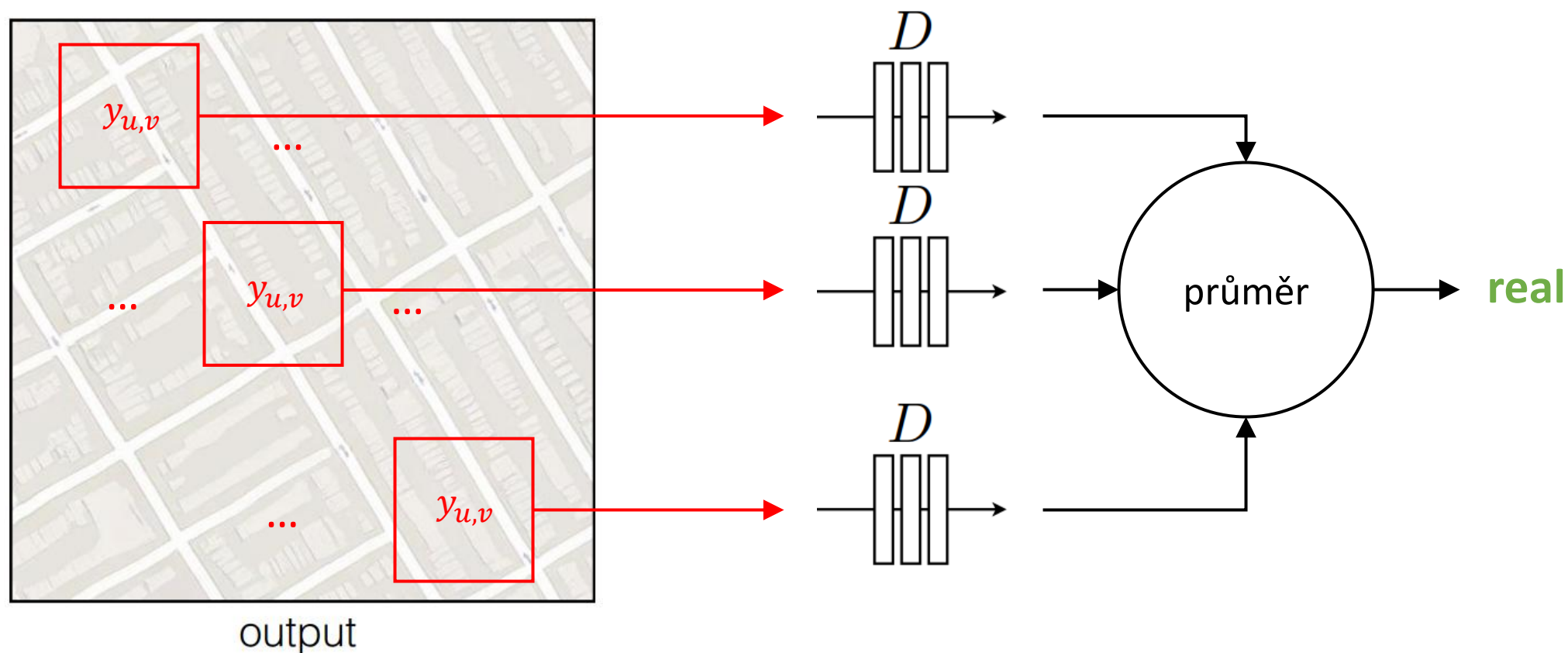


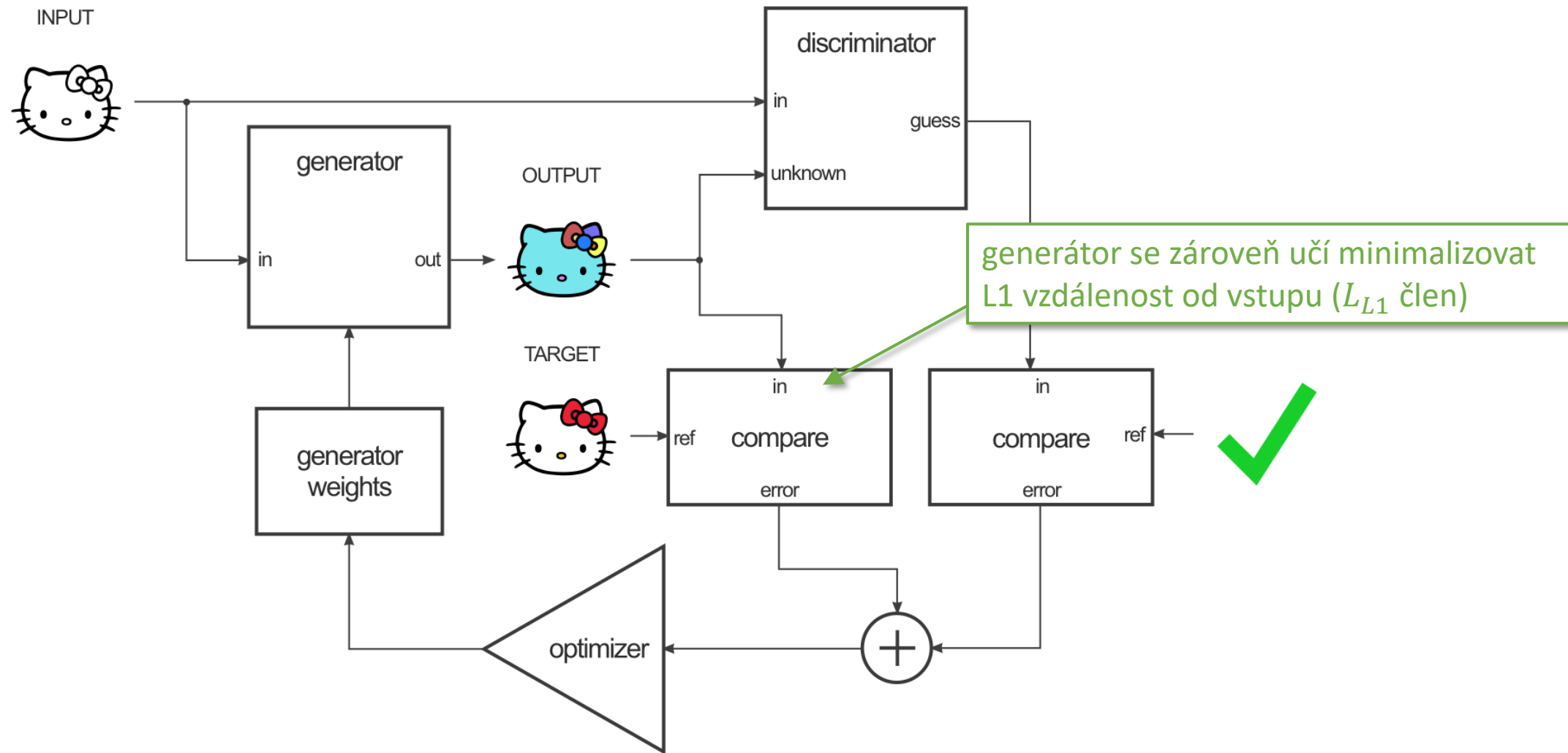
Figure 3: Two choices for the architecture of the generator. The “U-Net” [50] is an encoder-decoder with skip connections between mirrored layers in the encoder and decoder stacks.

pix2pix diskriminátor

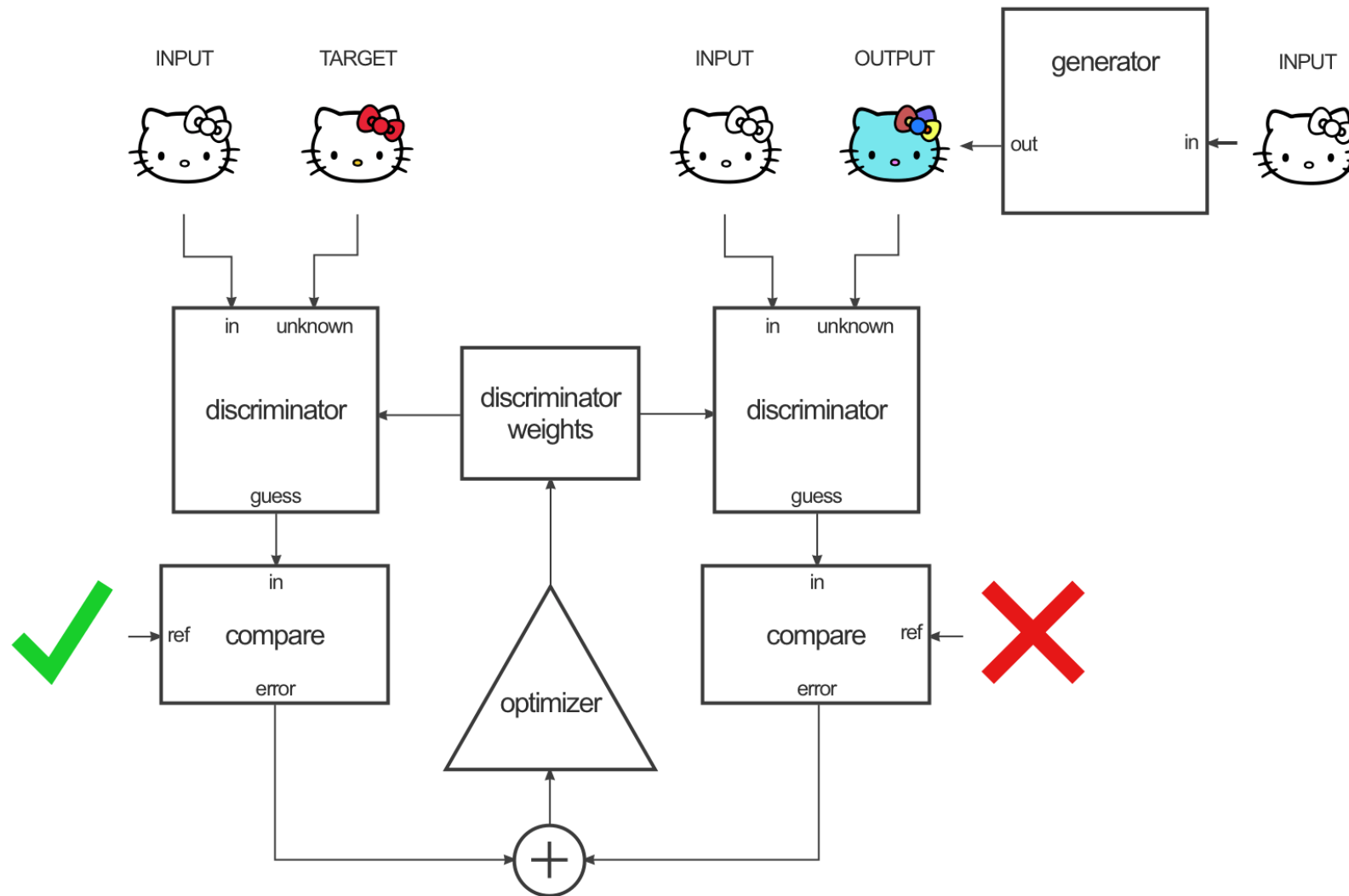
PatchGAN ... diskriminátor (konvoluční síť) namísto celého obrázku klasifikuje pouze jednotlivé patche a pak průměruje



pix2pix trénování generátoru



pix2pix trénování diskriminátoru



obrázek: <https://affinelayer.com/pix2pix/>

pix2pix (2016)



Figure 8: Example results on Google Maps at 512x512 resolution (model was trained on images at 256×256 resolution, and run convolutionally on the larger images at test time). Contrast adjusted for clarity.

pix2pix (2016)

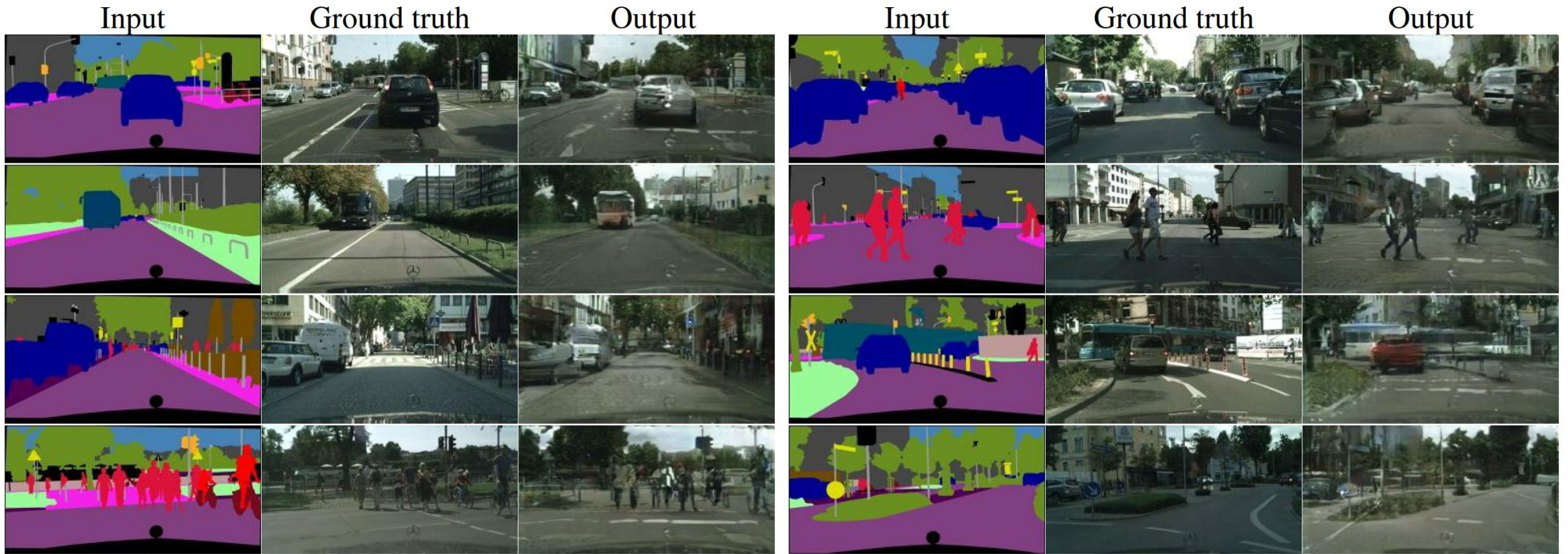


Figure 13: Example results of our method on Cityscapes labels→photo, compared to ground truth.

pix2pix (2016)



Figure 15: Example results of our method on day→night, compared to ground truth.

pix2pix (2016)



Figure 16: Example results of our method on automatically detected edges→handbags, compared to ground truth.

pix2pix (2016)



Figure 17: Example results of our method on automatically detected edges→shoes, compared to ground truth.

pix2pix (2016)

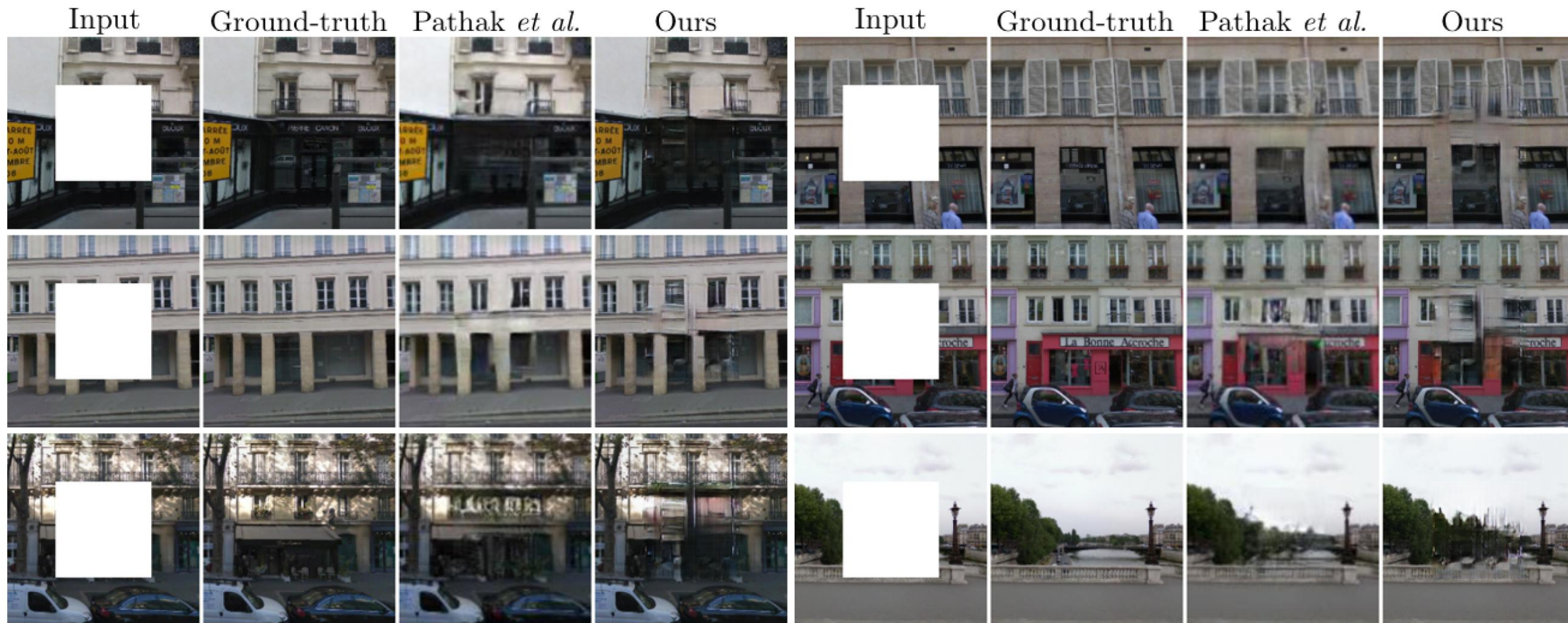


Figure 19: Example results on photo inpainting, compared to [43], on the Paris StreetView dataset [14]. This experiment demonstrates that the U-net architecture can be effective even when the predicted pixels are not geometrically aligned with the information in the input – the information used to fill in the central hole has to be found in the periphery of these photos.

pix2pix (2016)

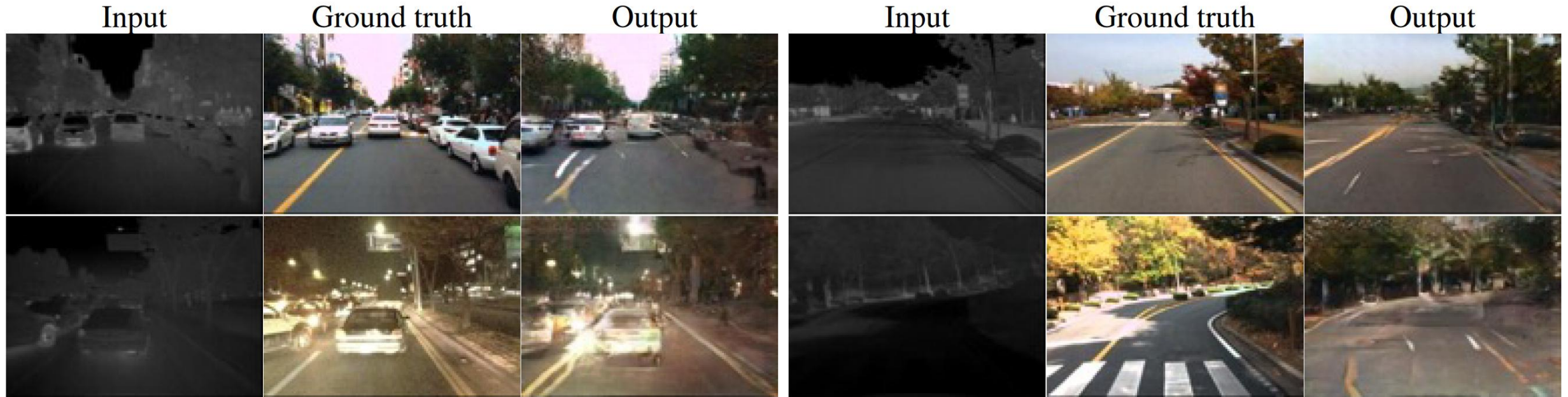
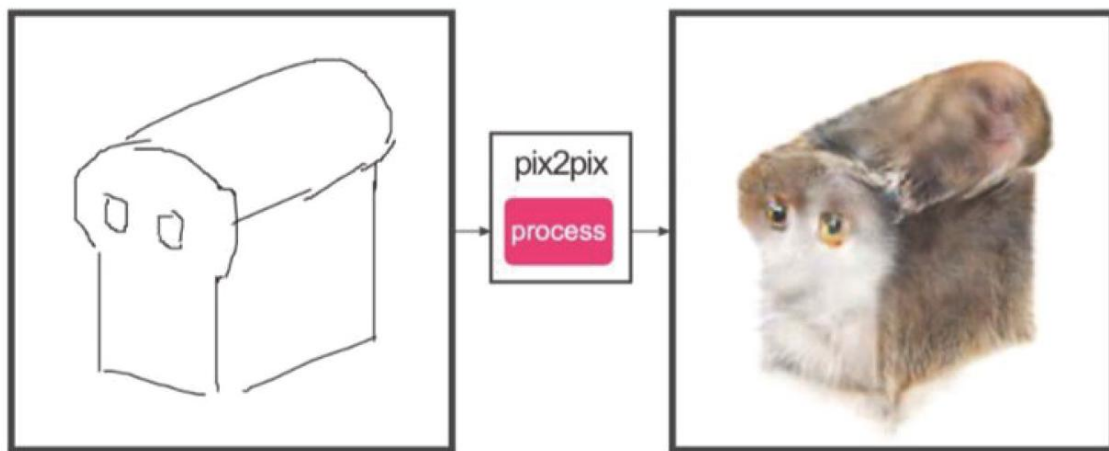


Figure 20: Example results on translating thermal images to RGB photos, on the dataset from [27].

pix2pix (2016)

#edges2cats by Christopher Hesse



sketch by Ivy Tsai

Background removal



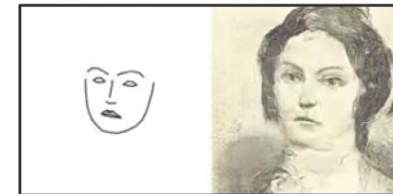
by Kaihu Chen

Palette generation



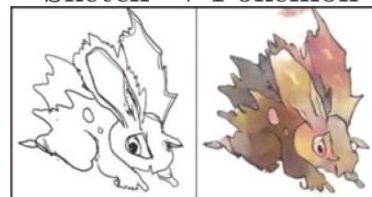
by Jack Qiao

Sketch → Portrait



by Mario Klingemann

Sketch → Pokemon



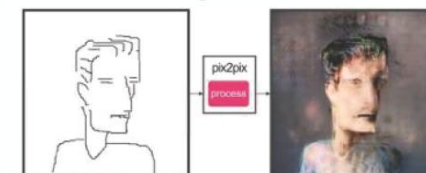
by Bertrand Gondouin

“Do as I do”



by Brannon Dorsey

#fotogenerator



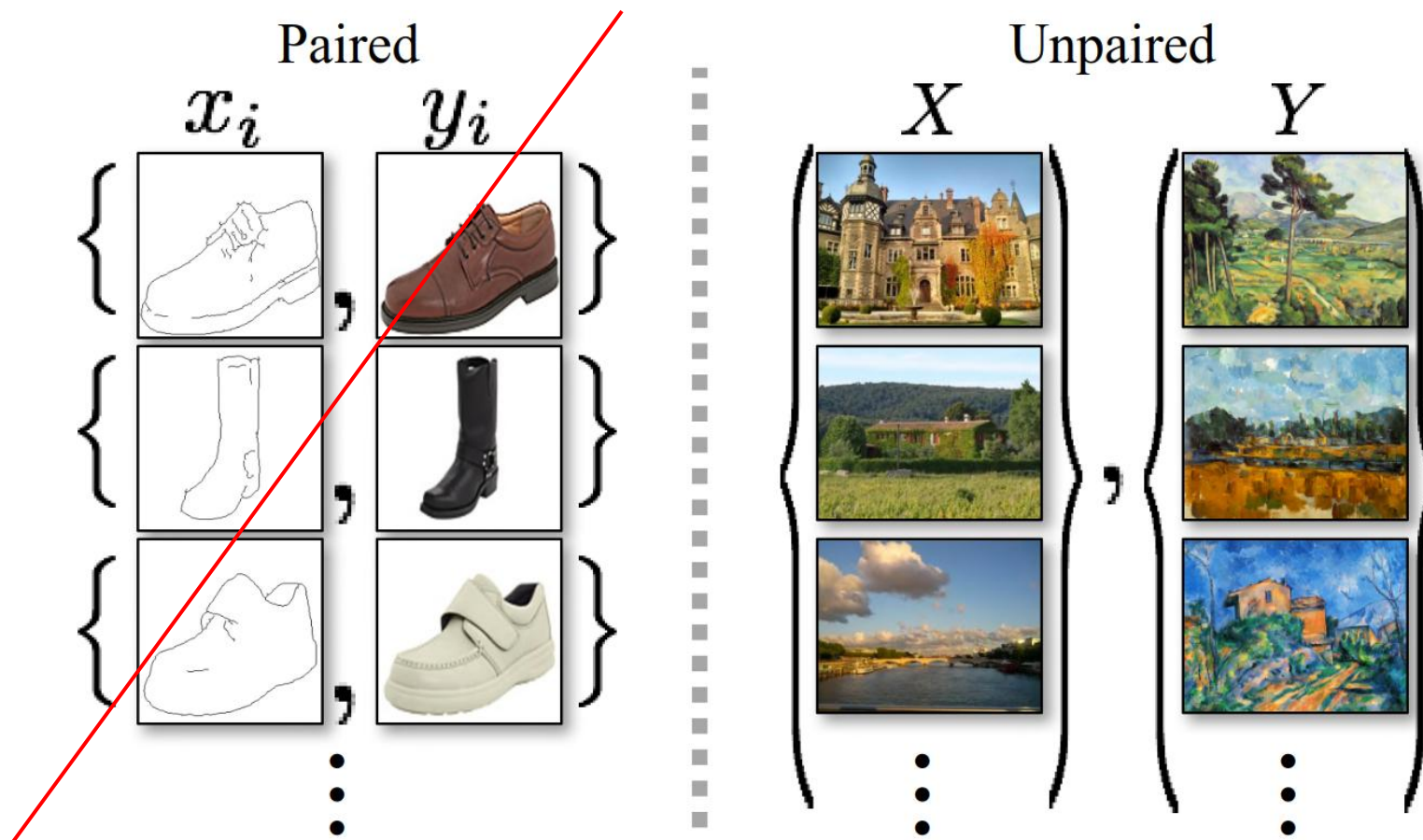
sketch by Yann LeCun

Figure 11: Example applications developed by online community based on our `pix2pix` codebase: *#edges2cats* [3] by Christopher Hesse, *Background removal* [6] by Kaihu Chen, *Palette generation* [5] by Jack Qiao, *Sketch → Portrait* [7] by Mario Klingemann, *Sketch → Pokemon* [1] by Bertrand Gondouin, “Do As I Do” pose transfer [2] by Brannon Dorsey, and *#fotogenerator* by Bosman et al. [4].

online demo: <https://affinelayer.com/pixsrv/>

CycleGAN (2017)

- [Zhu et al.: Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks](#)
- Nepotřebuje dataset párů (vstup, výstup), stačí nespárované kolekce
- Např. sada letních obrázků a druhá sada zimních → **unsupervised***



CycleGAN (2017)

- Obsahuje 2 generátory G, F a 2 diskriminátory D_X, D_Y
- Vstup se převádí z domény X do domény Y a **zároveň i zpět** → cyklický loss
 - Ve výstupu generátoru musí zůstat informace vstupu a nemůže se naučit generovat pouze náhodně

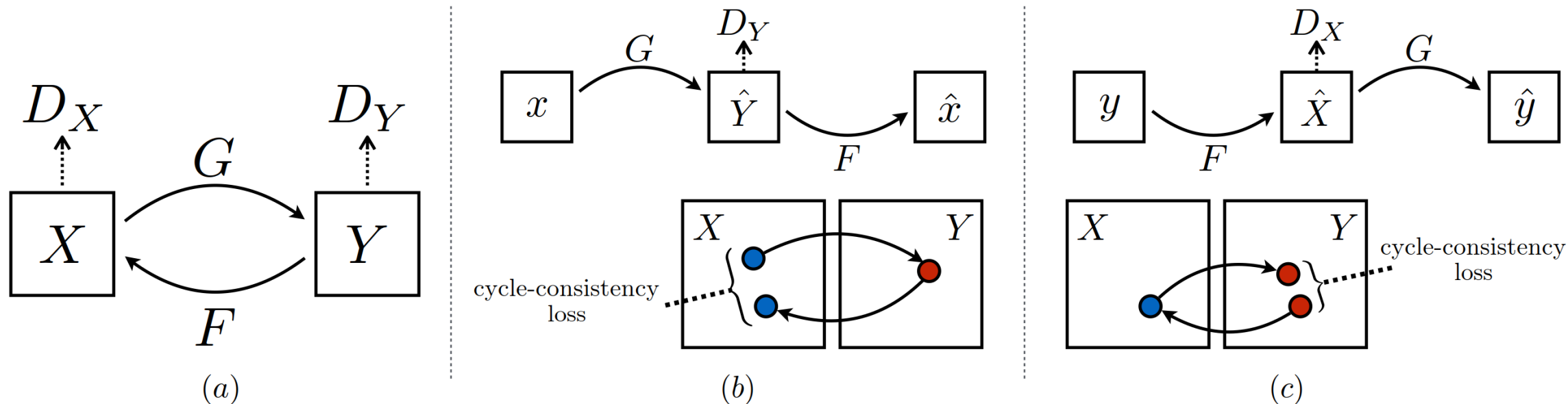


Figure 3: (a) Our model contains two mapping functions $G : X \rightarrow Y$ and $F : Y \rightarrow X$, and associated adversarial discriminators D_Y and D_X . D_Y encourages G to translate X into outputs indistinguishable from domain Y , and vice versa for D_X and F . To further regularize the mappings, we introduce two *cycle consistency losses* that capture the intuition that if we translate from one domain to the other and back again we should arrive at where we started: (b) forward cycle-consistency loss: $x \rightarrow G(x) \rightarrow F(G(x)) \approx x$, and (c) backward cycle-consistency loss: $y \rightarrow F(y) \rightarrow G(F(y)) \approx y$

CycleGAN (2017)

- GAN loss

$$L_{GAN}(G, D_Y) = \mathbb{E}_{y \sim Y} [\log D_Y(y)] + \mathbb{E}_{x \sim X} [\log (1 - D_Y(G(x)))] \quad \leftarrow \boxed{\text{foto} \rightarrow \text{malba}}$$

$$L_{GAN}(F, D_X) = \mathbb{E}_{x \sim X} [\log D_X(x)] + \mathbb{E}_{y \sim Y} [\log (1 - D_X(F(y)))] \quad \leftarrow \boxed{\text{malba} \rightarrow \text{foto}}$$

- Cyklický loss

$$L_{cyc}(G, F) = \underbrace{\mathbb{E}_{x \sim X} [\|F(G(x)) - x\|_1]}_{\text{true foto vs rekonstr. foto}} + \underbrace{\mathbb{E}_{y \sim Y} [\|G(F(y)) - y\|_1]}_{\text{true malba vs rekonstr. malba}}$$

- Celkový loss

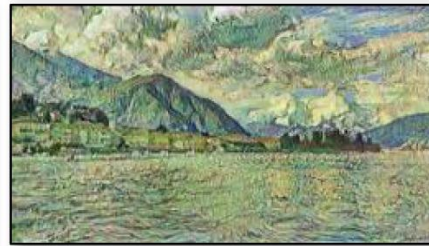
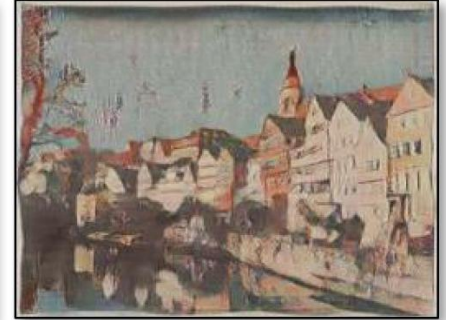
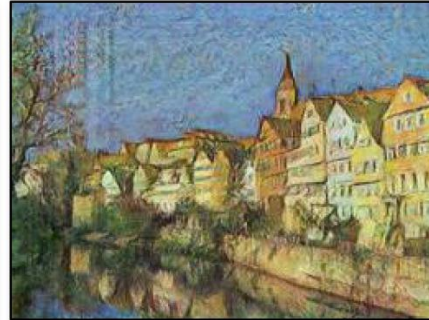
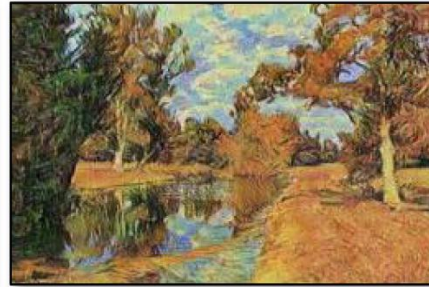
$$\begin{aligned} L_{CycleGAN}(G, F, D_X, D_Y) = & L_{GAN}(G, D_Y) \\ & + L_{GAN}(F, D_X) \\ & + L_{cyc}(G, F) \end{aligned}$$

CycleGAN (2017)

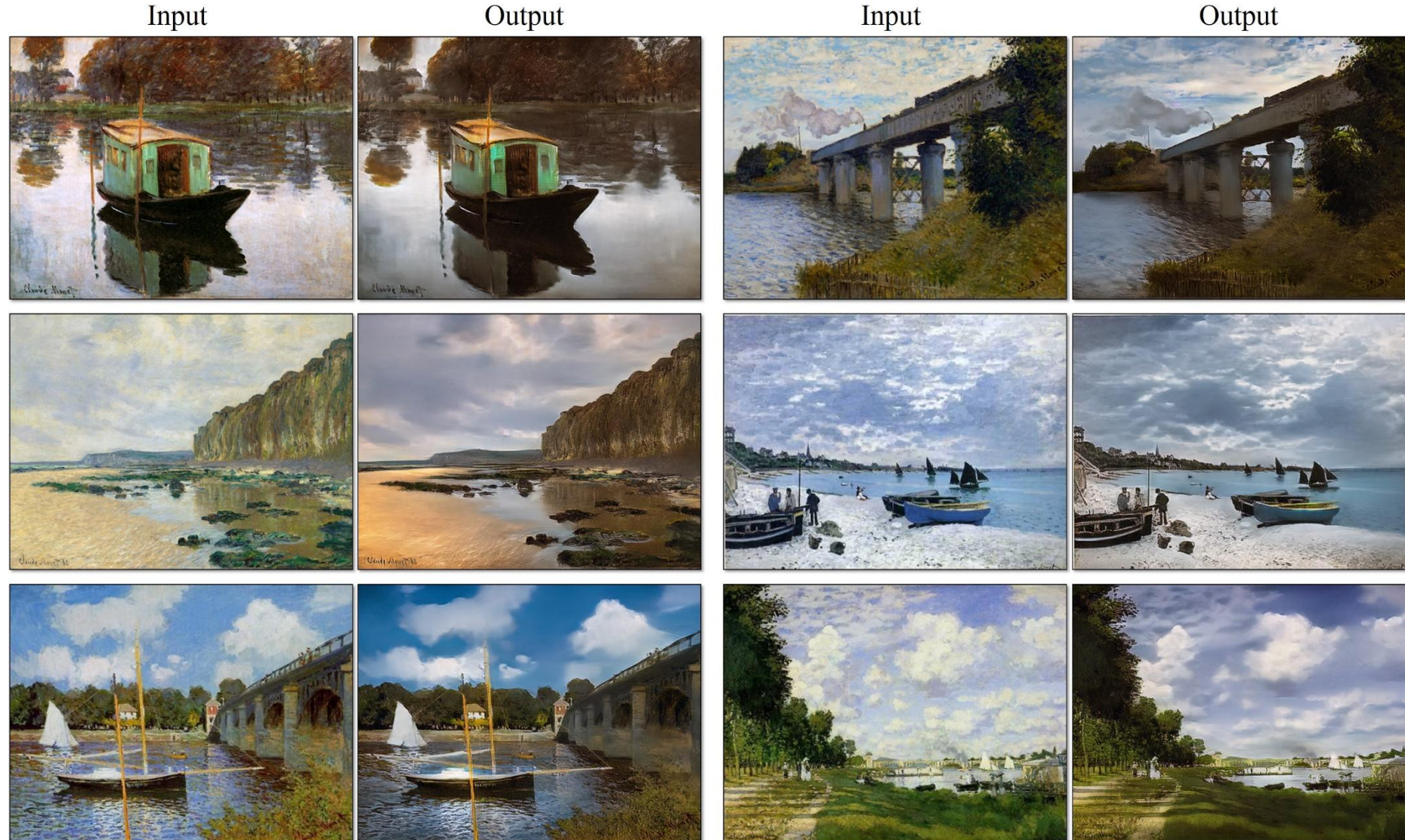


Figure 7: Different variants of our method for mapping labels $\leftrightarrow$ photos trained on cityscapes. From left to right: input, cycle-consistency loss alone, adversarial loss alone, GAN + forward cycle-consistency loss ($F(G(x)) \approx x$), GAN + backward cycle-consistency loss ($G(F(y)) \approx y$), CycleGAN (our full method), and ground truth. Both *Cycle alone* and *GAN + backward* fail to produce images similar to the target domain. *GAN alone* and *GAN + forward* suffer from mode collapse, producing identical label maps regardless of the input photo.

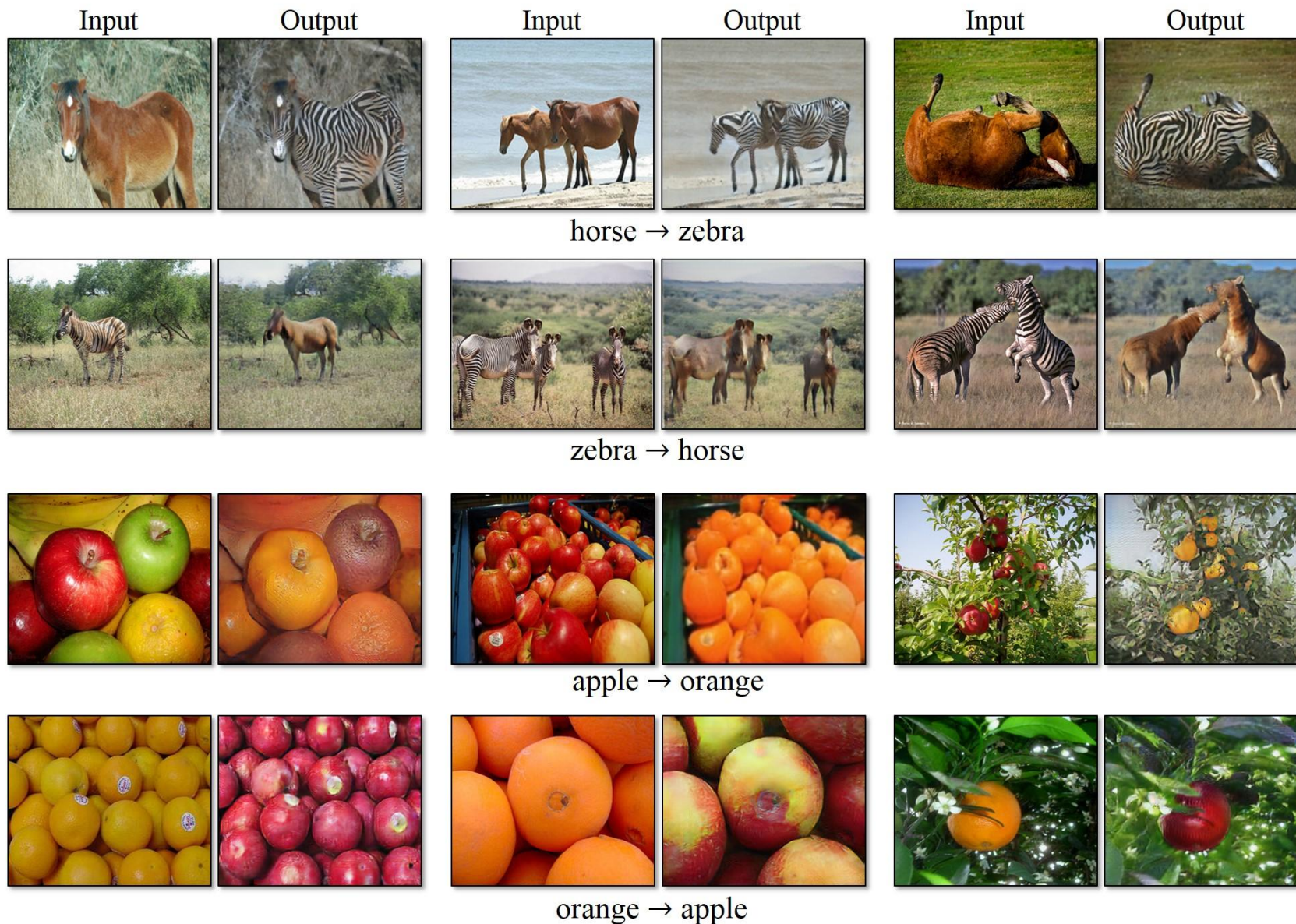
CycleGAN (2017): Style transfer



CycleGAN (2017): Monet → fotografie



CycleGAN (2017): Transmogrifikace



CycleGAN (2017): Transmogrifikace



<https://junyanz.github.io/CycleGAN/>

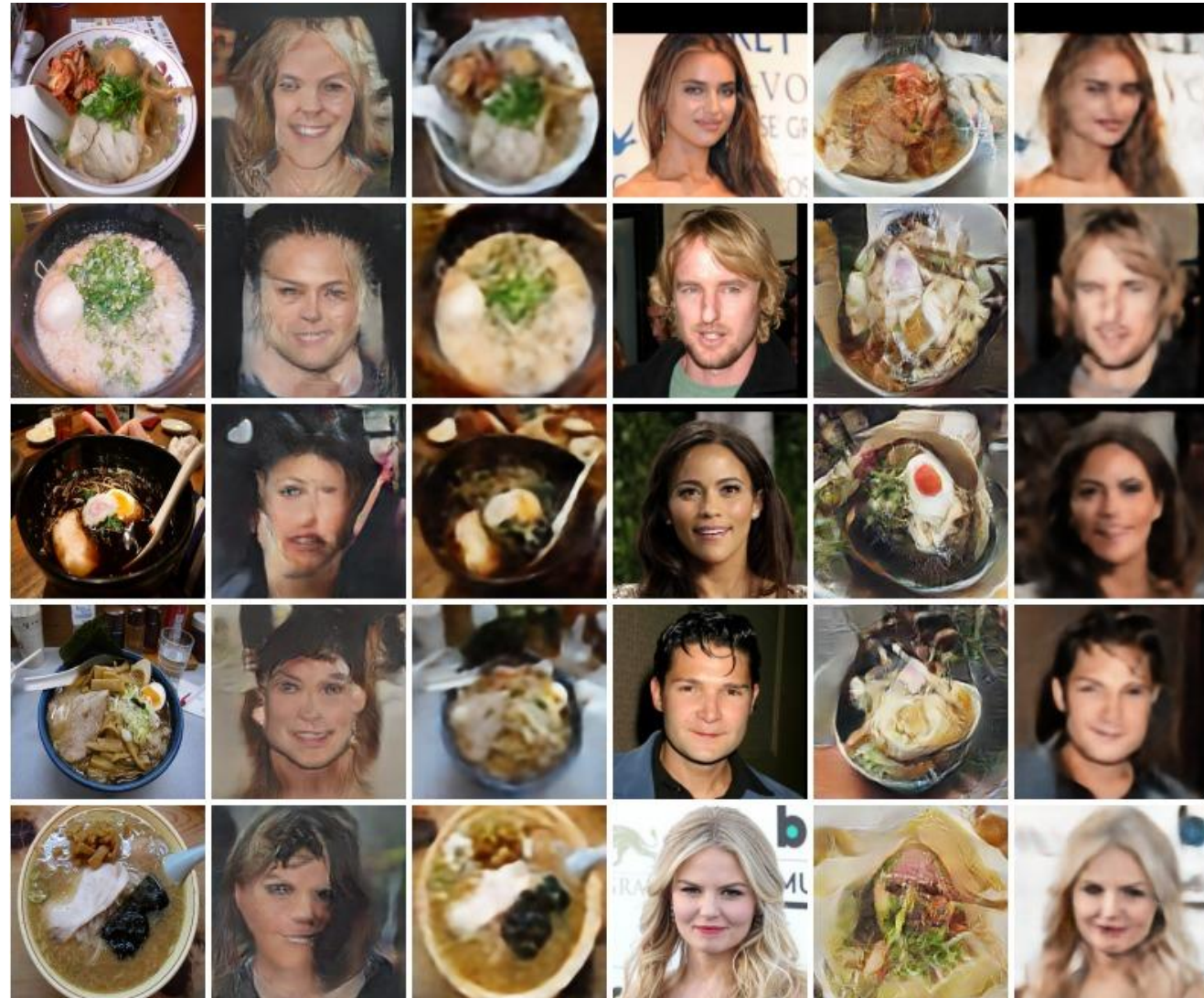
CycleGAN (2017): Když to není dokonalé



CycleGAN (2017): Když to není dokonalé

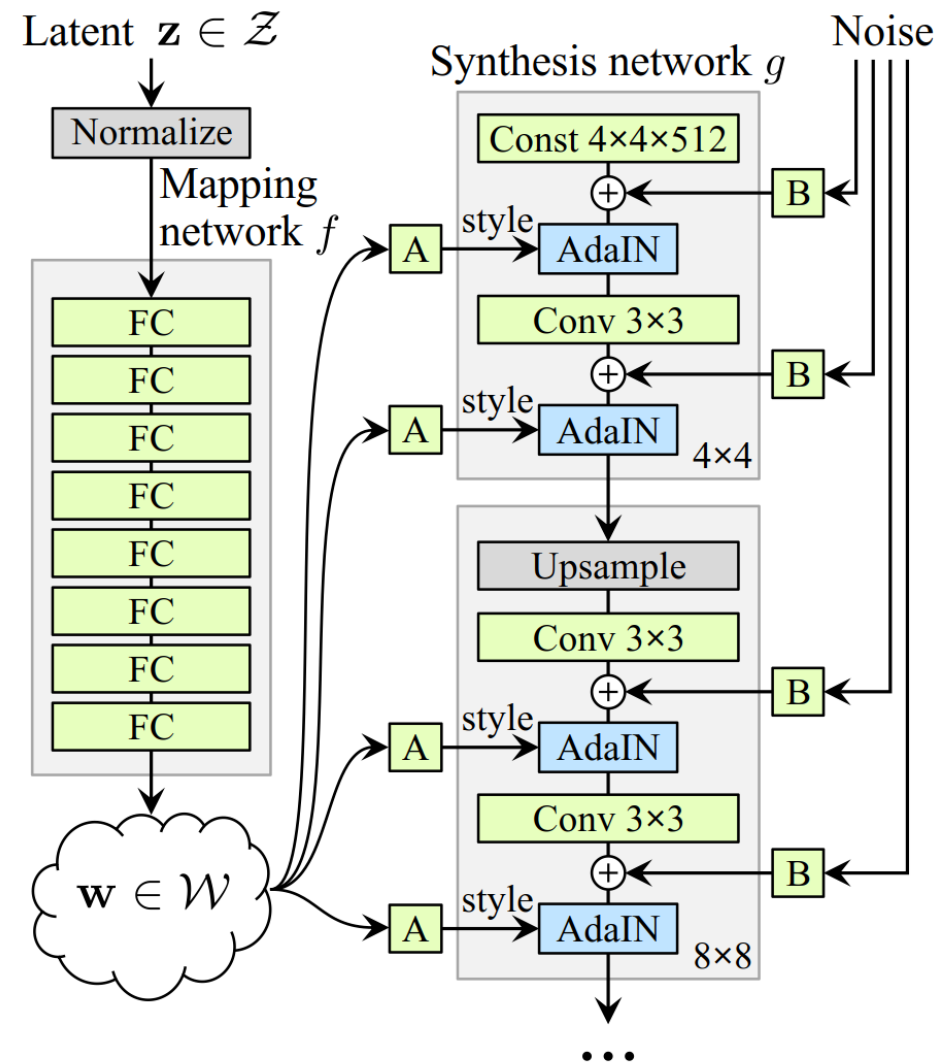


CycleGAN (2017): Face to Ramen



StyleGAN (2018)

- [Karras et al.: A Style-Based Generator Architecture for Generative Adversarial Networks](#)
- Unsupervised generování, nepodmíněná GAN
- Mapovací síť f převádějící prostor z na „styl“ w
- Progresivní zvyšování rozlišení



(b) Style-based generator

Instance normalization

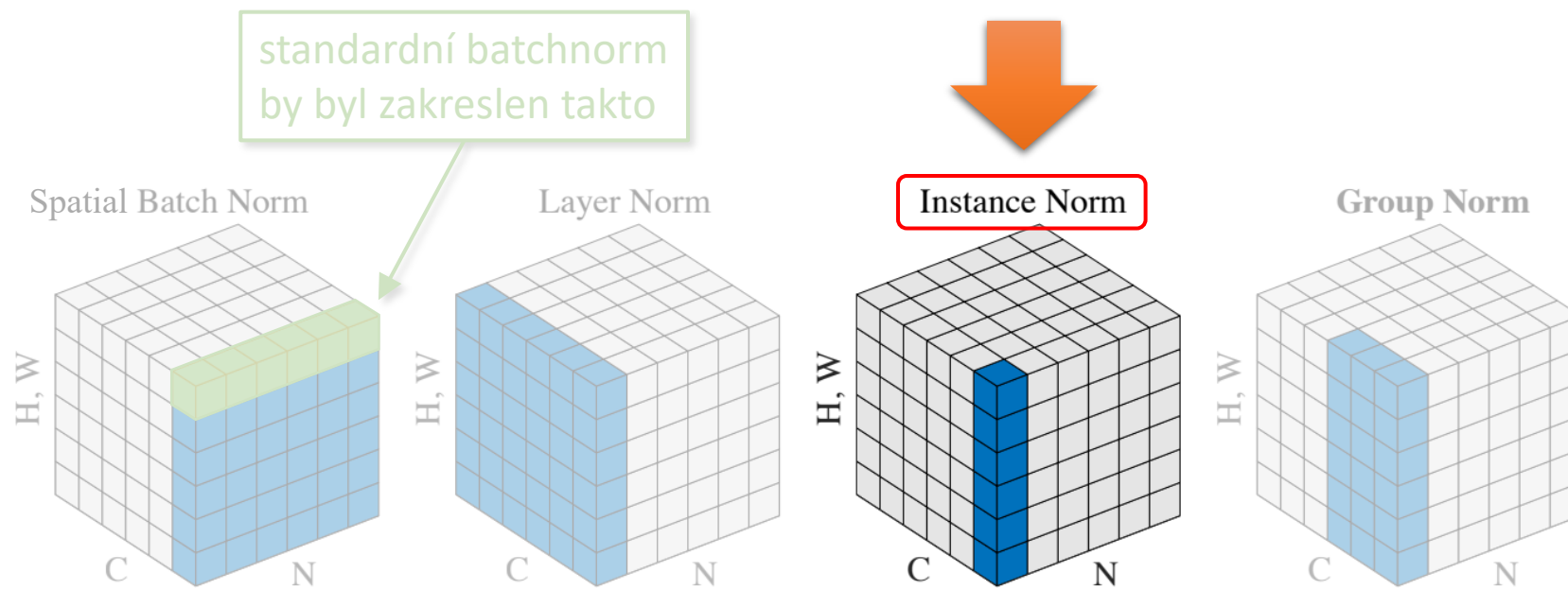


Figure 2. **Normalization methods.** Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

StyleGAN (2018)

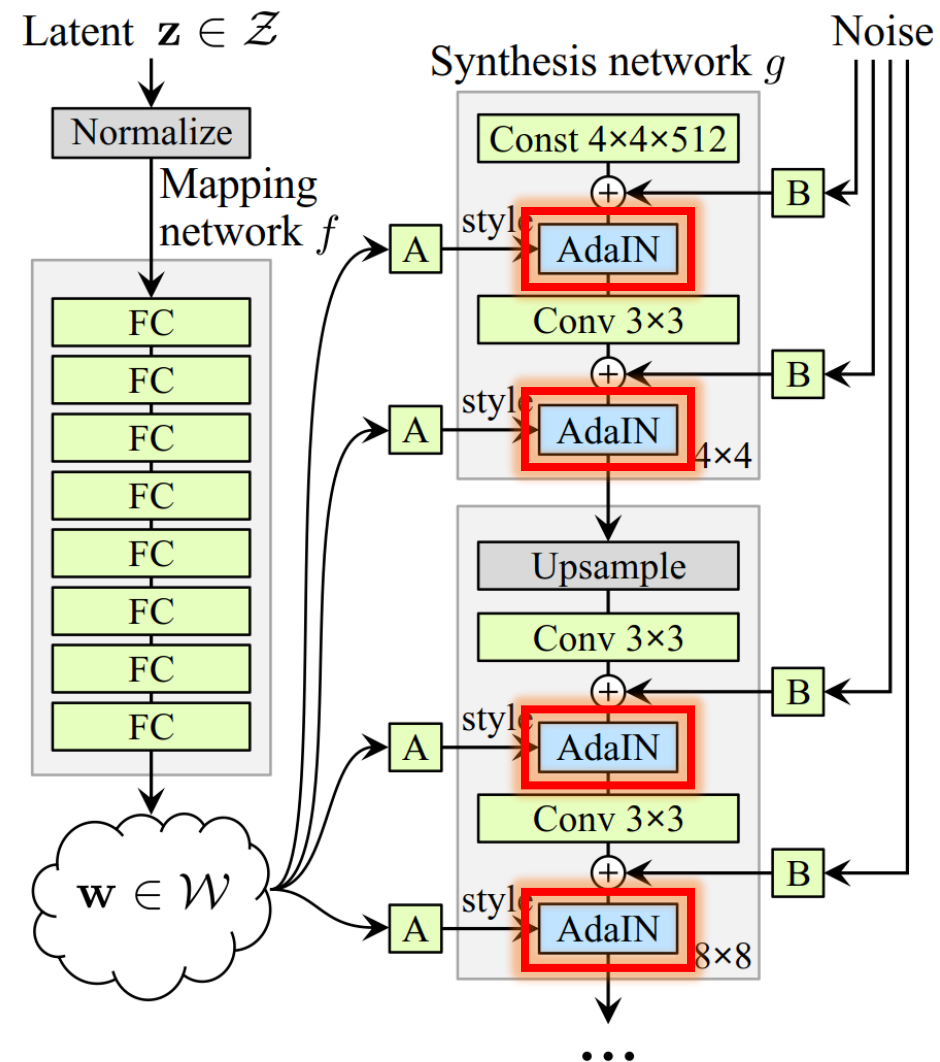
- Adaptive instance normalization

$$\text{AdaIN}(x, y) = y_{s,i} \frac{x_i - \mu(x_i)}{\sigma(x_i)} + y_{b,i}$$

x ... feature mapa předchozí vrstvy generátoru

y ... vytvořeno z w – scale a bias pro každý kanál

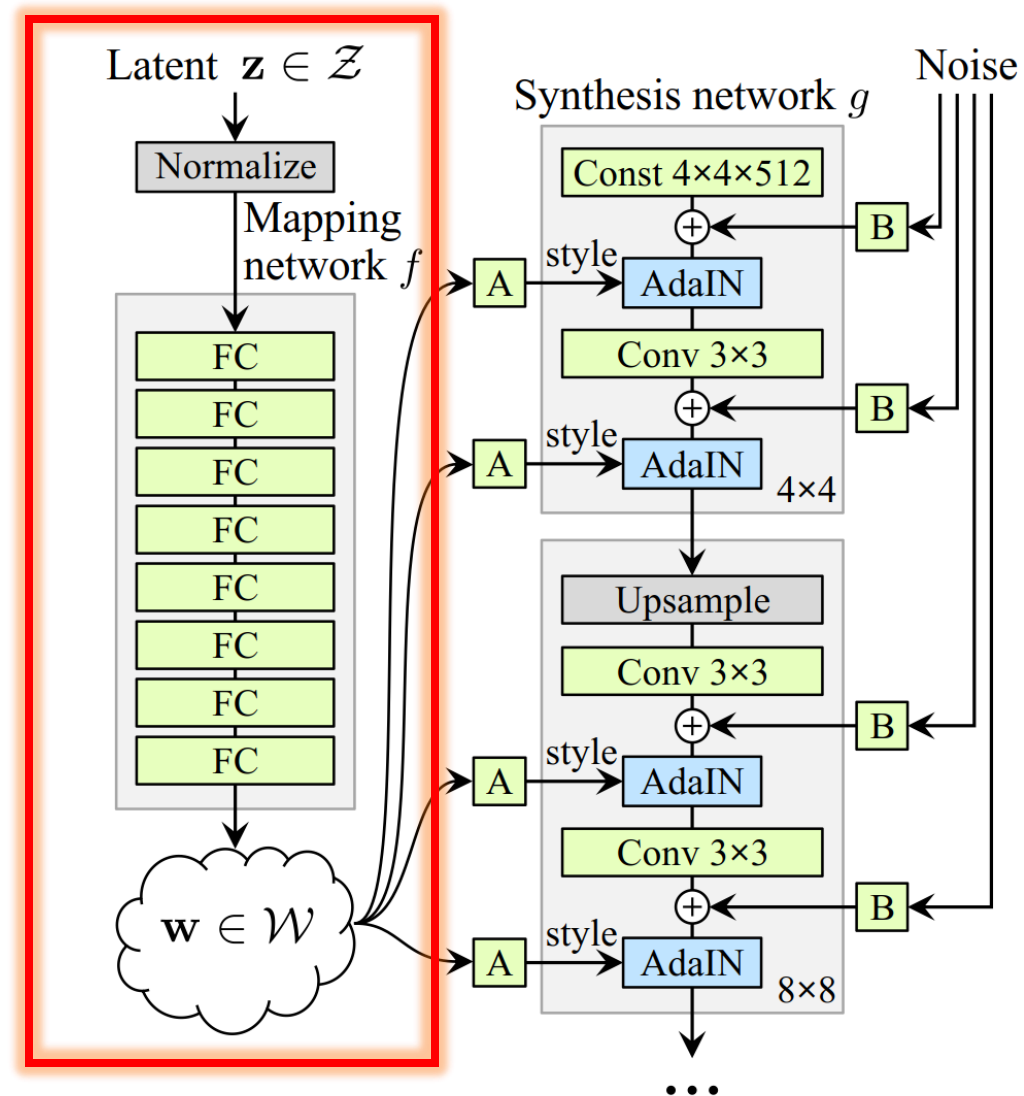
- Funguje jako styl
 - V původním článku na style transfer použity gram matice = kovariance kanálů
 - Transfer stylu ale funguje i s jinými statistikami, např. průměr a rozptyl (variance)
 - Tj. pokud vynormalizujeme kanály feature map z obr. A tak, aby měly stejný průměr a rozptyl jako mapy z jiného obrázku B, přeneseme styl z B do A
 - Zde ovšem cílové statistiky nepřenášíme z jiného obrázku, nýbrž generujeme mapovací síť $y = A(f(z))$



(b) Style-based generator

StyleGAN (2018)

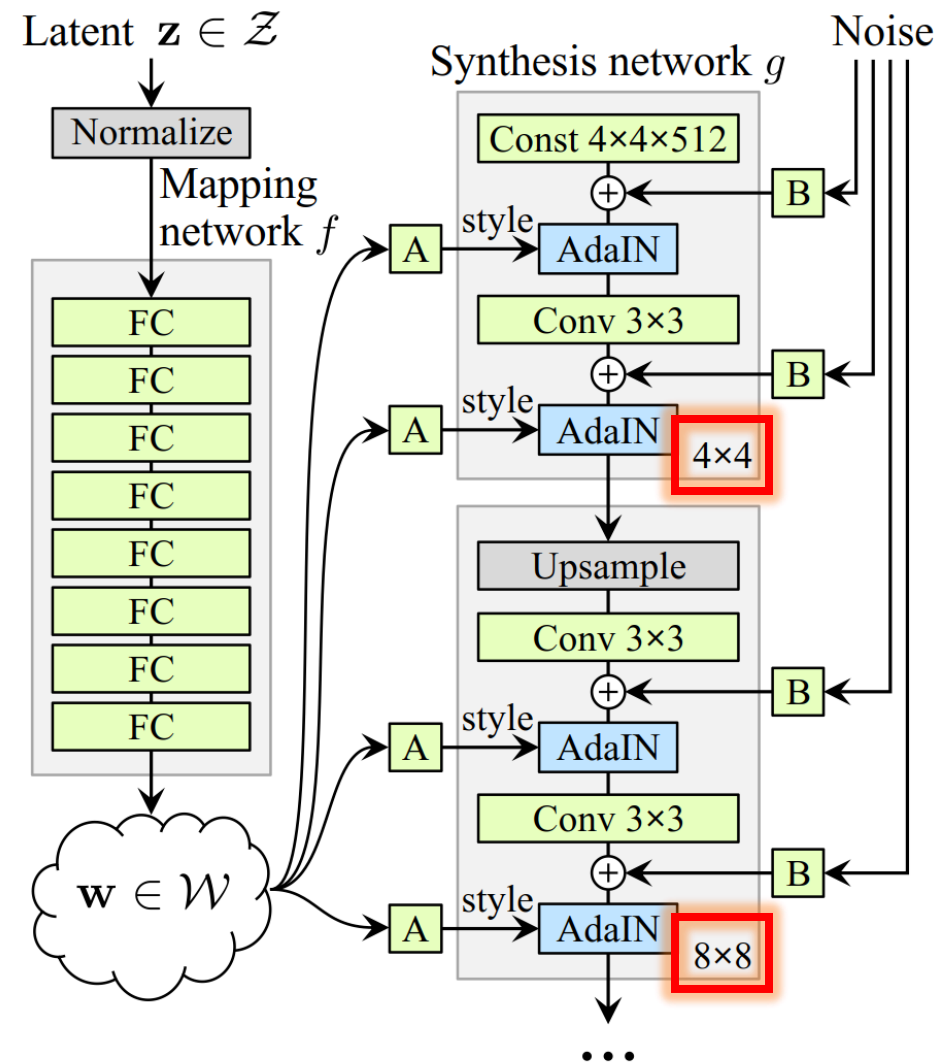
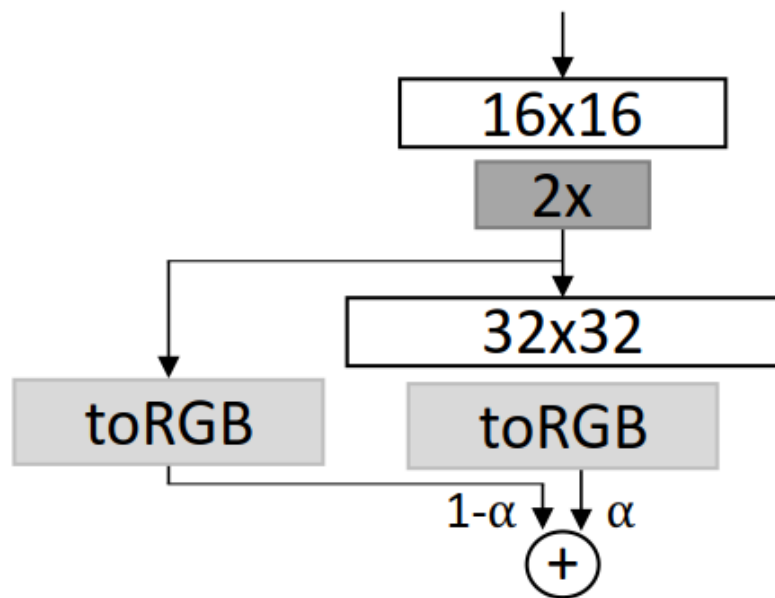
- Latentní proměnné $z \sim p(z)$ nejsou na začátku generátoru, ale nejprve převedeny na mezi-proměnné w
- Ty jsou pak dodány do různých vrstev skrze afinní transformaci (bloky A) $y = Aw + b$
- Výstup y pak funguje jako styl



(b) Style-based generator

StyleGAN (2018)

- Trénování po vrstvách
- Nejprve naučí generovat 4x4, počkají až zkonvertuje, poté přidají Upsample vrstvu 8x8 a opakují až do 1024x1024
- Není to „natvrdo“, ale ResNet-like prolínáním:



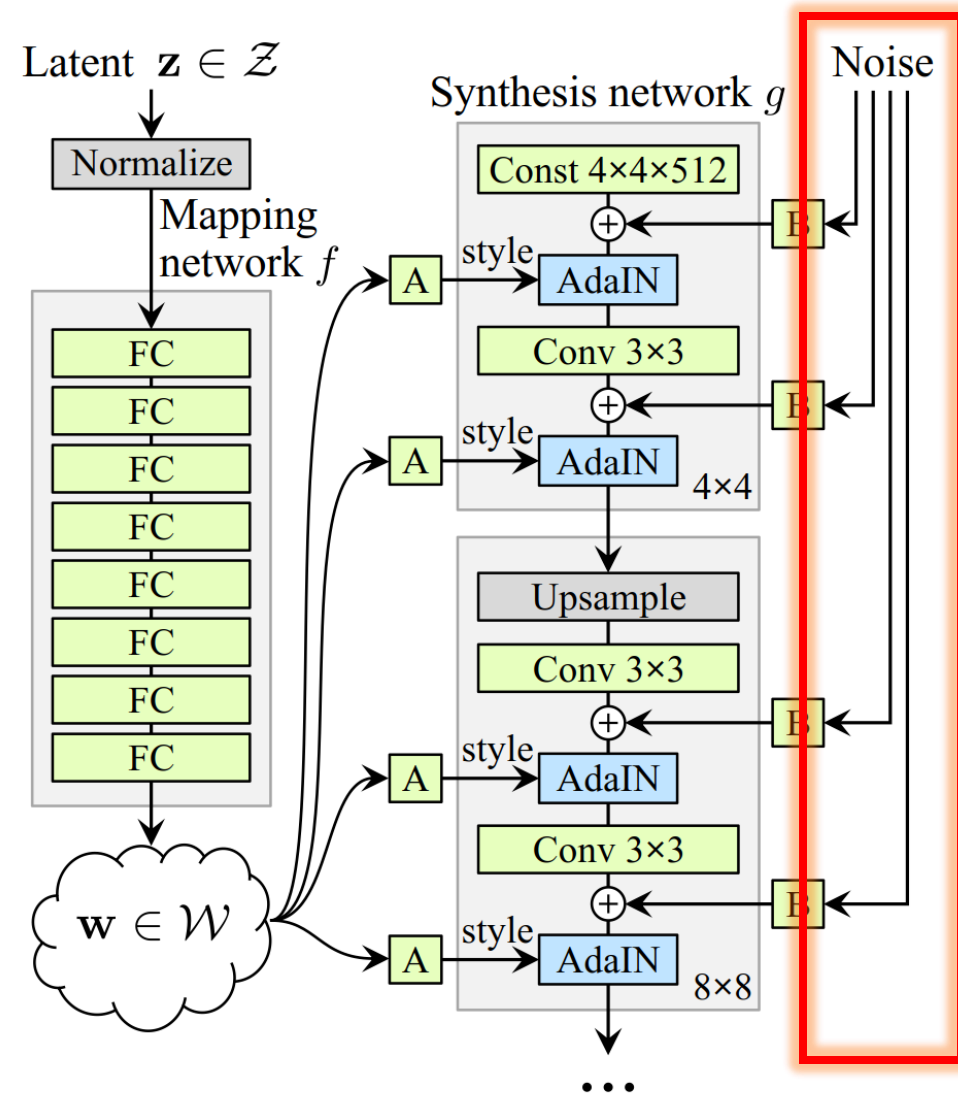
(b) Style-based generator

StyleGAN (2018)



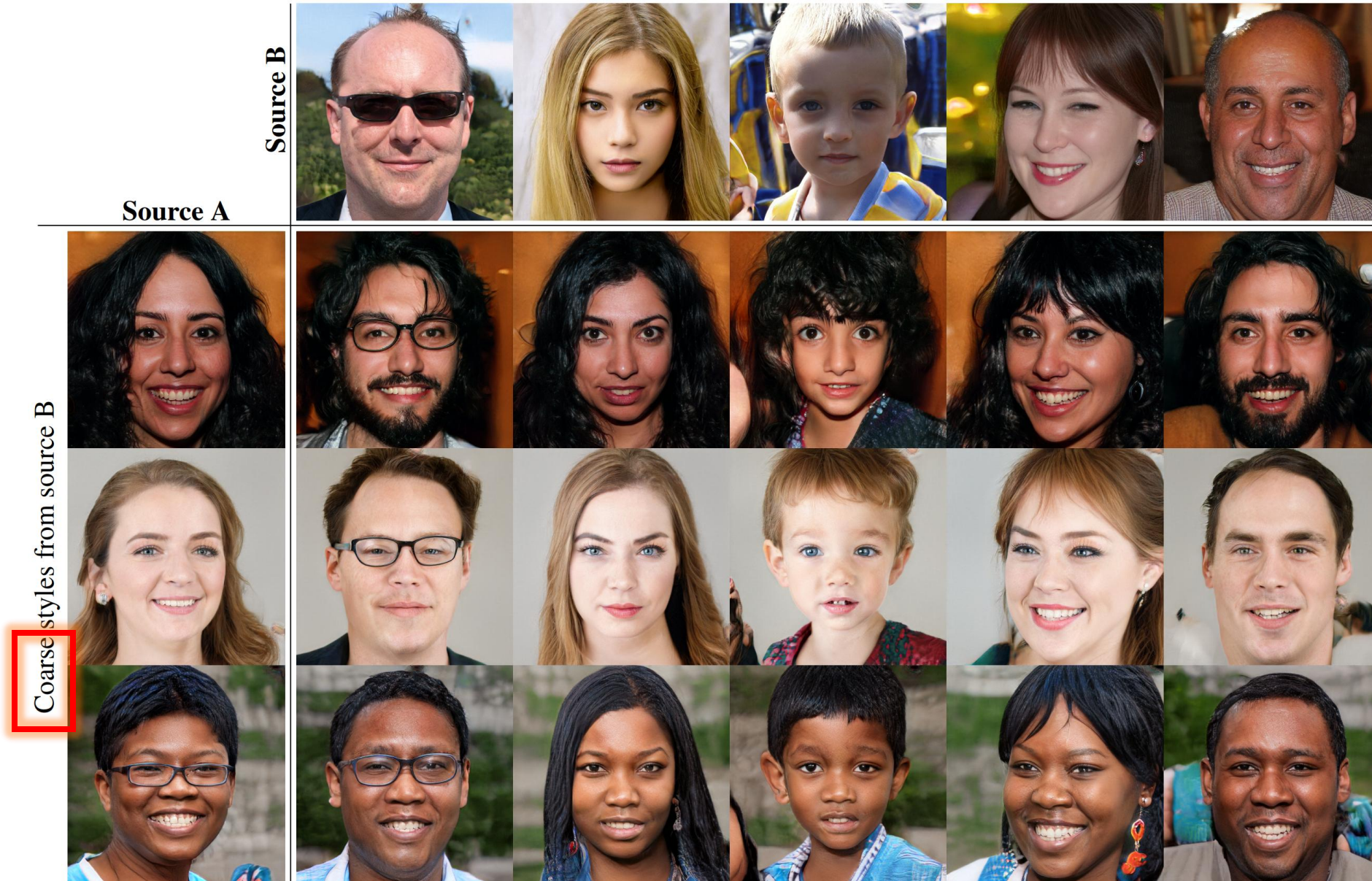
StyleGAN (2018)

- Šum dodán pro mírnou stochasticitu
- Ovlivní pouze drobné detaily

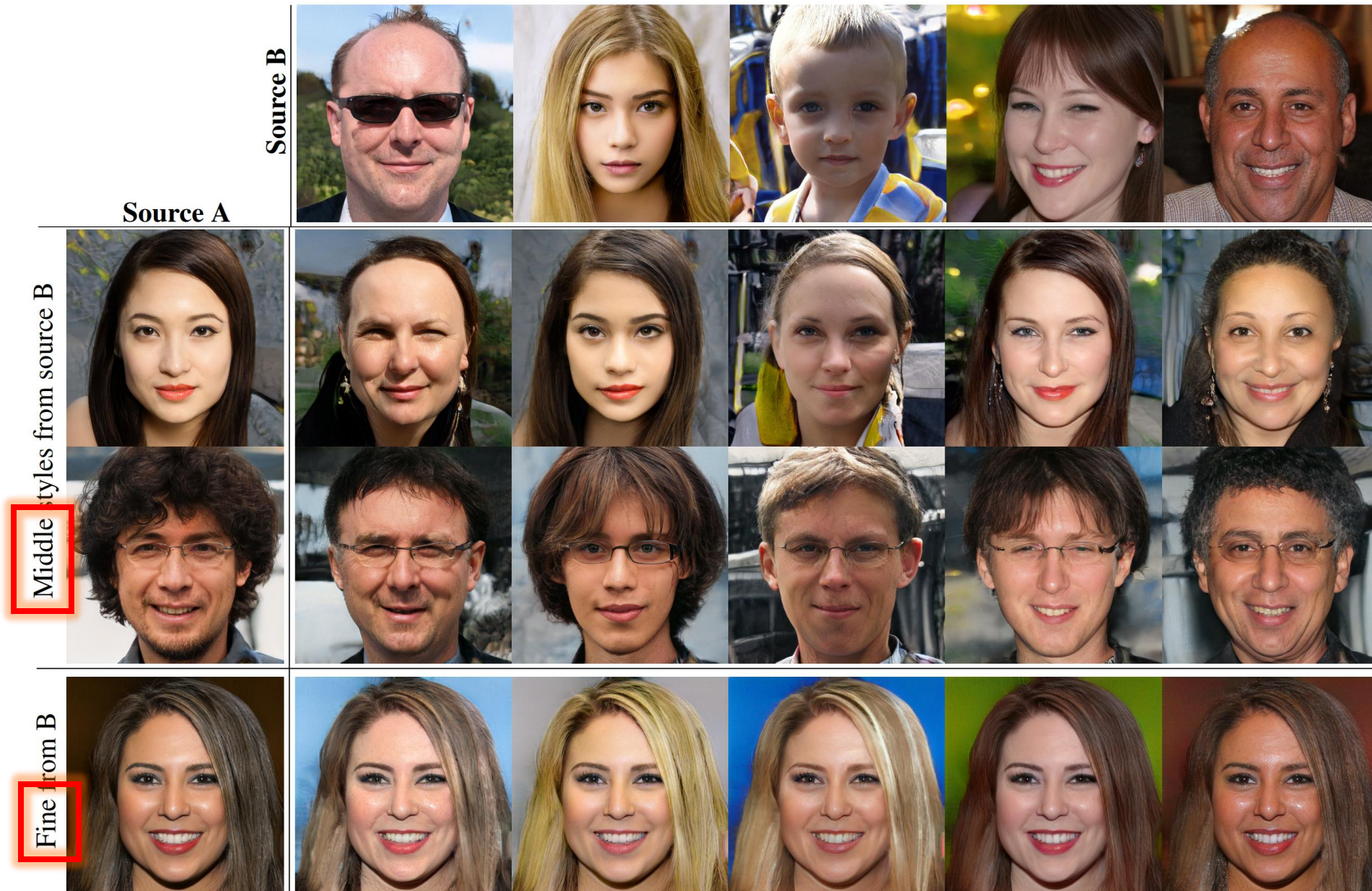


(b) Style-based generator

StyleGAN (2018): mixování stylů



StyleGAN (2018): mixování stylů



StyleGAN2 (2019)

- [Karras et al.: Analyzing and Improving the Image Quality of StyleGAN](#)
- Vylepšená verze
- Nevyužívá progresivního zvyšování rozlišení, trénuje najednou
- Snaží se především opravit artefakty



StyleGAN2 (2019)

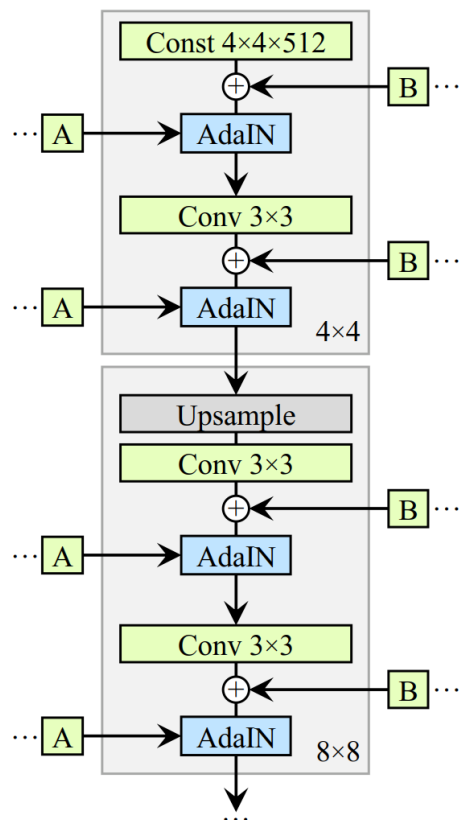
- Progresivní zvyšování způsobuje „fázové“ artefakty



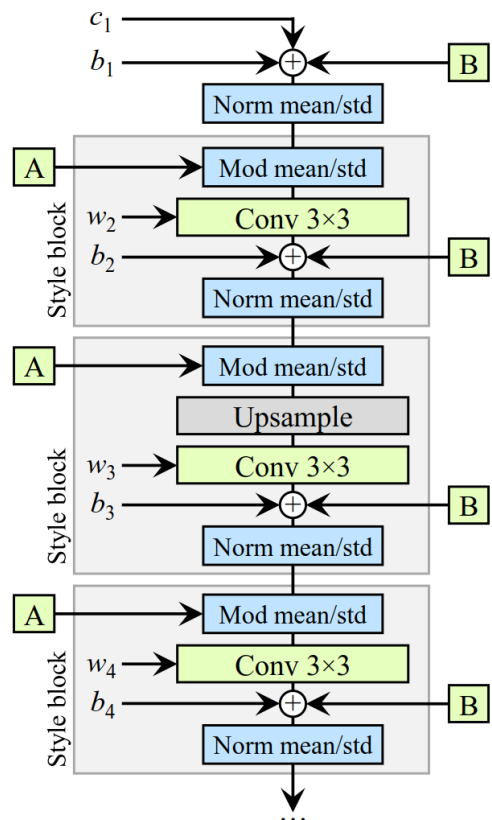
Figure 6. Progressive growing leads to “phase” artifacts. In this example the teeth do not follow the pose but stay aligned to the camera, as indicated by the blue line.

StyleGAN2 (2019)

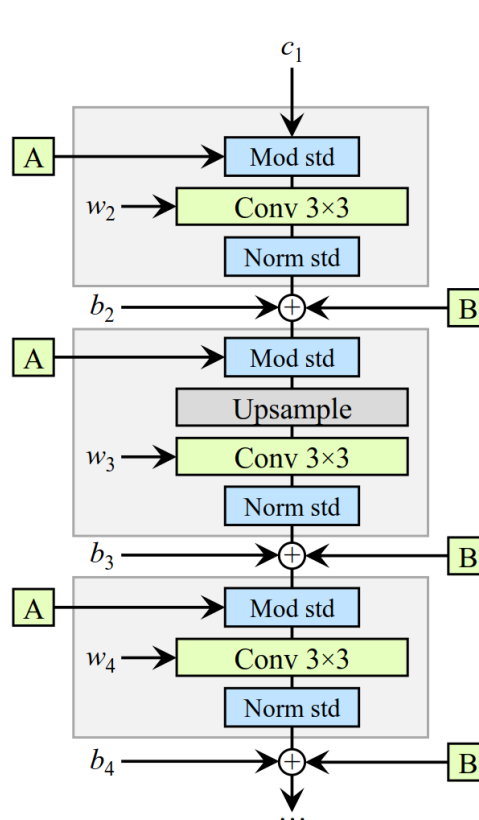
- Změny v architektuře generátoru
- Normalizace zahrnuta přímo do vah konvolučního filtru



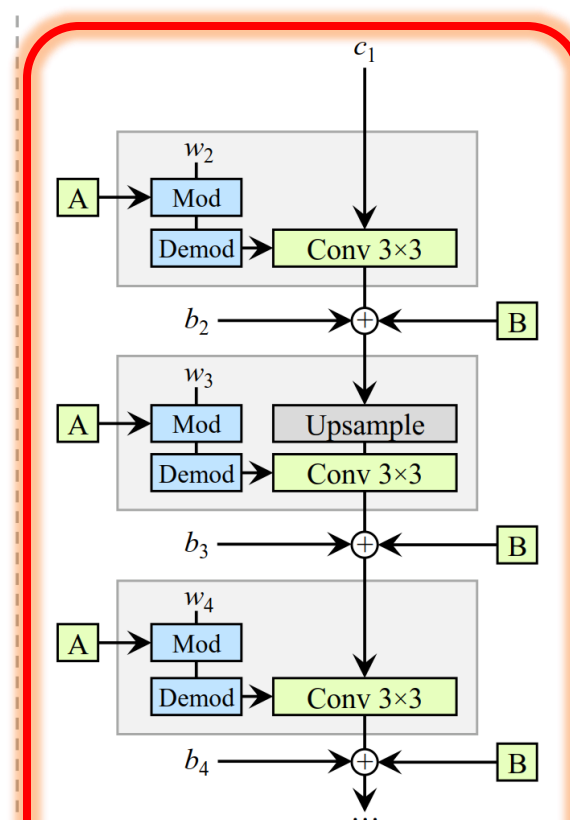
(a) StyleGAN



(b) StyleGAN (detailed)



(c) Revised architecture



(d) Weight demodulation

StyleGAN2 (2019)



<https://thispersondoesnotexist.com/>

StyleGAN2 (2019)



<https://thiscatdoesnotexist.com/>

StyleGAN2 (2019)

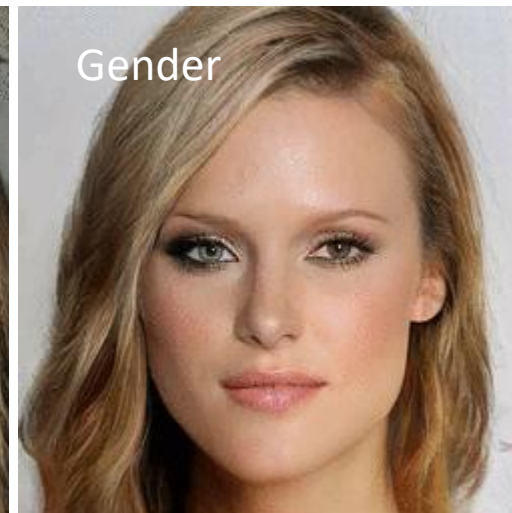
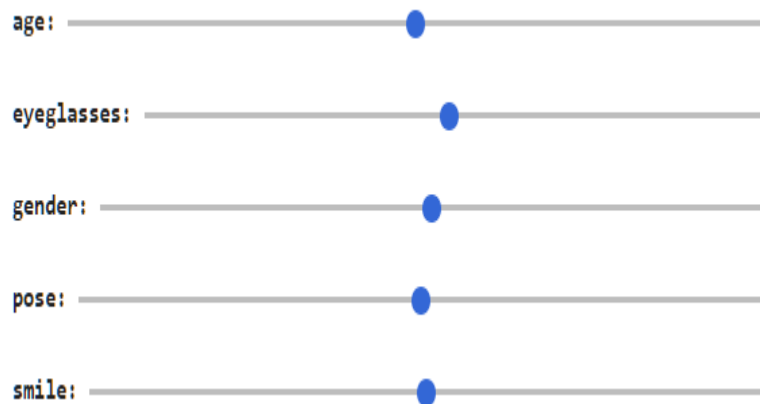


<https://thishorsedoesnotexist.com/> (thank god)

InterFaceGAN (2019)

- [Shen et al.: Interpreting the Latent Space of GANs for Semantic Face Editing](#)
- Prostor latentních proměnných z dobře odpovídá faktorům variability v reálném světě

Směr jednotlivých os zjištěn
natrénováním lineárního SVM nad
vektory z



InterFaceGAN (2019)

- Pro úpravu existující fotky potřebujeme nějak zjistit odpovídající „kód“ z
- Např. optimalizací: učení vektoru z tak, aby feature mapy nebo pixely byly co nejpodobnější
- Nebo síť, která přímo převádí obrázek $\rightarrow$ vektor z
- Vznikne rekonstrukce $\rightarrow$ na ní provedeme úpravy $\rightarrow$ profit

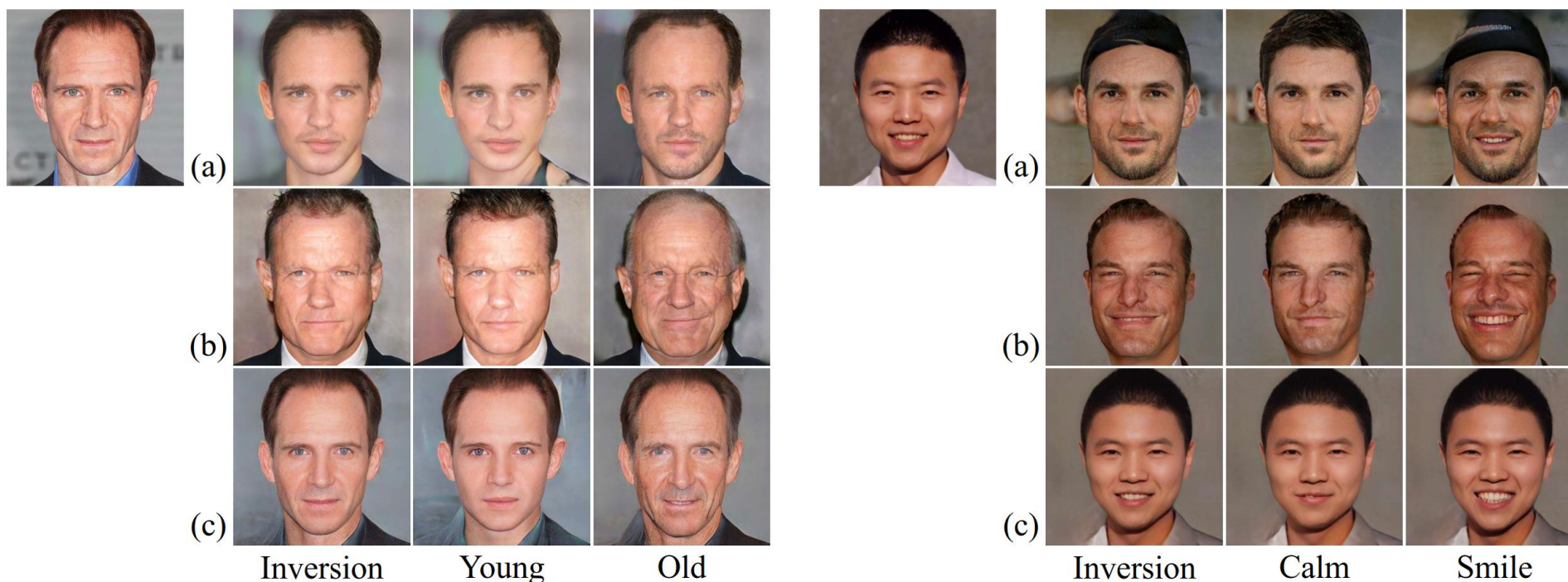


Figure 10: Manipulating real faces with respect to the attributes age and gender, using the pre-trained PGGAN [21] and StyleGAN [22]. Given an image to edit, we first invert it back to the latent code and then manipulate the latent code with InterFaceGAN. On the top left corner is the input real face. From top to bottom: (a) PGGAN with optimization-based inversion method, (b) PGGAN with encoder-based inversion method, (c) StyleGAN with optimization-based inversion method.